**NELLAI NADAR COLLEGE OF ENGINEERING AND TECHNOLOGY**

**Approved AICTE| Affiliated to Anna University| Accredited by NBA**

**Nedungundram, Chennai-127**

# DEPARTMENT OF
# Artificial Intelligence & Data Science

## Project name

WhatNext Vision Motors:

Shaping the Future of Mobility with Innovation and Excellence

**Team ID:SWTID-2026-4131**

**Team Members**

**KEERTHIKA S (Team leader)**

**JAYASUDHA M** (member)

**MAHESWARI S** (member)

**SARUKESH G** (member)

**SAI SURYA B** (member)

## User story

WhatsNext Vision Motors, a pioneering force in the automotive industry, is dedicated to transforming the mobility sector with innovative technology and solutions that prioritize customer needs. The company has embarked on an ambitious Salesforce project with the core objective of enhancing the customer experience and streamlining its operational processes.
At the heart of this project is the improvement of the customer ordering process. The system is designed to automatically suggest the nearest dealer location to customers based on their address. This feature is intended to significantly enhance the convenience and efficiency of the ordering experience, making it more customer-friendly and reducing the time and effort required from the customer's end.
The project also addresses a common issue in the automotive industry: stock availability. The system includes a mechanism that prevents customers from placing orders for vehicles that are out of stock. This proactive approach ensures that customers can only create orders for vehicles that are currently available, thus avoiding potential confusion and disappointment that may arise from stock unavailability. This feature not only enhances customer satisfaction but also improves the accuracy of the company's order fulfillment process.
Furthermore, the project incorporates a scheduled process for updating the status of bulk order records. This automated process is designed to update the order status based on stock availability. If a vehicle is out of stock at the time of order placement, the system will update the order status to 'Pending.' On the other hand, if the vehicle is in stock, the system will update the status to 'Confirmed.' This ensures that all orders are accurately reflected in terms of their fulfillment status, providing clear and transparent communication to customers regarding the status of their orders.
The implementation of this Salesforce project at WhatsNext Vision Motors is expected to yield several benefits. It aims to create a more efficient ordering system that reduces the potential for errors and improves the overall service provided to customers. By streamlining the ordering process and ensuring accurate stock availability, the company can enhance customer satisfaction and loyalty.
Moreover, the project is expected to contribute to operational efficiency by reducing the administrative burden on staff. By automating certain processes, employees can focus on more strategic tasks that require human intervention and expertise. This not only improves the overall productivity of the company but also allows for a more agile response to market demands and customer needs.

### 1.INTRODUCTION

WhatNext Vision Motors
Tagline: Shaping the Future of Mobility with Innovation and Excellence

**1.1 project overview:**

WhatNext Vision Motors is a next-generation mobility initiative engineered to redefine how people, goods, and cities move. By converging advanced electrification, autonomous intelligence, connected infrastructure, and sustainable design, the project delivers transportation solutions that are smarter, cleaner, and inherently user-centric. WhatNext Vision Motors isn't just building vehicles; we're architecting an integrated mobility ecosystem for the decades ahead.

## Vision & Mission

Vision: To pioneer a seamless, sustainable, and intelligent mobility future that empowers communities, enhances quality of life, and safeguards the planet.

Mission: To accelerate the global transition to zero-emission, connected, and automated mobility through relentless R&D, strategic partnerships, and uncompromising standards of safety, performance, and excellence.

## Core Focus Areas

- Next-Gen Powertrains : High-efficiency electric, hydrogen, and hybrid architectures optimized for range, performance, and lifecycle sustainability.
- Autonomous & Connected Systems : AI-driven perception, V2X communication, and predictive navigation for safer, more efficient mobility.
- Smart Mobility Integration : Mobility-as-a-Service (MaaS) platforms, fleet optimization, and seamless multimodal transit connectivity.
- Sustainable Engineering : Circular manufacturing, low-impact materials, carbon-neutral operations, and end-of-life vehicle recycling.
- Human-Centric Design : Intuitive interfaces, adaptive accessibility features, and personalized in-vehicle experiences.

## What Sets Us Apart

WhatNext Vision Motors operates at the intersection of mobility, software, and sustainability. Our modular vehicle platforms, open-data connectivity framework, and agile development cycles enable rapid iteration, scalable production, and continuous over-the-air upgrades. Excellence is embedded in every phase: from ethically sourced supply chains to rigorous safety validation and post-deployment analytics.

## Roadmap & Impact

Phase 1 : Prototype validation, pilot deployments in smart-city corridors, and MaaS platform beta launch.
Phase 2 : Scaled manufacturing, fleet partnerships, and integration with public transit networks.
Phase 3 : Global commercialization, carbon-neutral operations, and full ecosystem interoperability.

By bridging cutting-edge technology with real-world usability, WhatNext Vision Motors aims to reduce urban congestion, cut transportation emissions, and deliver equitable, future-ready mobility for all.

**Data Management-Objects**

-Purpose : Acts as the persistent system of record for all integration credentials, their metadata, and lifecycle history.
-Configuration: Create a custom object (e.g., `Integration_Credential__c`). Enable Audit Fields, Field History Tracking, and Change Data Capture. Add lookup/master-detail fields to link credentials to specific endpoints, business units, or integration flows.
-Security:Set Organization-Wide Defaults (OWD) to `Private`. Enforce CRUD/FLS exclusively through Permission Sets. Avoid public sharing rules. Enable Shield Platform Encryption if storing regulated secrets.

**Data Management-Fields**

- Purpose: Captures the precise attributes, tokens, endpoints, and configuration parameters required for each credential.
- Configuration:
  - `Username__c` (Text)
  - `Secret_Token__c` (Encrypted Text)
  - `Endpoint_URL__c` (URL)
  - `Status__c` (Picklist: `Active`, `Expired`, `Rotating`, `Revoked`)
  - `Expiry_Date__c` & `Last_Rotated__c` (DateTime)
  - `Rotation_Frequency__c` (Picklist/Number)
- Security: Enforce strict Field-Level Security (FLS). Apply validation rules for format, length, and URL schemes. Mask encrypted fields on page layouts and list views. Never store plaintext passwords or keys.

**Data Management-Tabs**

-Purpose: Provides a dedicated, admin-facing interface for manual credential oversight, search, and bulk management.

- Configuration: Map a custom tab to the credential object. Configure default list views (e.g., *Expiring Soon*, *Active Integrations*, *Audit Log*). Add a compact layout and inline quick actions (e.g., *Trigger Rotation*, *Mark as Compromised*).
- Security:Restrict tab visibility to Integration Admins, Security Ops, or specific profiles via Permission Sets. Hide from standard business users to prevent accidental exposure or unauthorized edits.

**Data Management-App Manager**

-Purpose:Consolidates all credential-related components into a unified administrative workspace for streamlined governance, monitoring, and troubleshooting.
- Configuration:Build a custom app (e.g., *Credential Management Hub*) via Setup → App Manager. Include the credential tab, related flow builders, audit dashboards, and utility bar items (e.g., quick reference guides, log viewers). Set default landing page to a monitoring dashboard.
- Security:Control app assignment through Permission Sets. Apply session timeout policies and IP range restrictions if needed. Keeps credential management isolated from standard sales/service navigation.

**Automation**

-Purpose:Manages the declarative lifecycle of credentials, ensuring timely updates, compliance checks, and proactive alerts without manual intervention.
- Record-Triggered Flows: Auto-update status on creation, log initial setup steps, enforce mandatory field completion.
- Scheduled Flows: Run daily to flag credentials expiring in 30/14/7/1 days, trigger rotation workflows, or archive revoked records.
- Approval Processes: Require managerial or security team sign-off for production credential changes or new endpoint registrations.
- Security:Flows execute under the context of the initiating user or a dedicated integration user. Pass sensitive values using secure flow variables or reference `Named Credentials`. All flow paths and decisions are logged for audit compliance.

**Apex and Batch Class**

- Purpose: Handles complex, secure, or high-volume programmatic operations that exceed declarative limits or require external API interactions.
- Configuration:
  - `BatchApex` for bulk validation, mass rotation, or cleanup of expired credentials.
  - `Queueable`/`@future` for asynchronous token refreshes or external API handshakes.
  - `@InvocableMethod` to expose secure logic to Flows or external orchestration tools.
  - Implement robust error handling, retry logic, and secure HTTP request construction.

- Security:Always use `with sharing` and `Security.stripInaccessible()` before queries/DML. Never hardcode secrets; retrieve them via `Named Credentials`, `External Credentials`, or `Auth.Callout`. Use `System.runAs()` only in test contexts. Sanitize all debug logs to prevent token leakage.

# Salesforce Implementation Specification: WhatsNext Vision Motors

## 1. Use Case Summary

WhatsNext Vision Motors requires a Salesforce-based CRM solution to streamline the vehicle ordering process, improve customer experience, and optimize operational efficiency. The system must automatically suggest the nearest dealer based on customer location, prevent orders for out-of-stock vehicles, and automatically update order statuses based on real-time inventory. Automated workflows will manage dealer assignment, test drive reminders, and inventory monitoring to reduce manual effort and ensure accurate order fulfillment.

## 2. Data Architecture & Object Configuration

### 2.1 Custom Objects

- Vehicle__c: Stores inventory records, specifications, and stock status.
- Dealer__c: Represents dealership locations, service areas, and contact details.
- Vehicle_Order__c: Tracks customer orders, fulfillment status, and assigned dealers.
- Test_Drive_Request__c (Optional/Related): Manages scheduled test drives and service follow-ups.

### 2.2 Key Fields & Data Types

Vehicle__c
- VIN__c (Text, External ID)
- Model_Year__c (Text)
- Stock_Status__c (Picklist: Available, Reserved, Out of Stock)
- Current_Quantity__c (Number)
- Dealer__c (Lookup to Dealer__c)
- Location__c (Geolocation)

Dealer__c
- Name (Standard)

- Address__c (Compound Address)
- Location__c (Geolocation)
- Service_Radius_Miles__c (Number)
- Is_Active__c (Checkbox)

Vehicle_Order__c
- Customer__c (Lookup to Contact/Lead)
- Vehicle__c (Lookup to Vehicle__c)
- Assigned_Dealer__c (Lookup to Dealer__c)
- Order_Status__c (Picklist: Pending, Confirmed, Fulfilled, Cancelled)
- Test_Drive_Date__c (Date/Time)
- Customer_Location__c (Geolocation, auto-populated from shipping address)

## 3. Process Automation Configuration

### 3.1 Prevent Order Placement for Out-of-Stock Vehicles
- Use a **Before-Save Record-Triggered Flow** on Vehicle_Order__c.
- Query the related Vehicle__c record to check Stock_Status__c and Current_Quantity__c.
- If Stock_Status__c equals "Out of Stock" OR Current_Quantity__c is less than or equal to zero, trigger a Flow Fault or Screen Error.
- Display a clear user message: "This vehicle is currently out of stock. Please select an available model or contact your dealer."

### 3.2 Auto-Assign Orders to the Nearest Dealer
- Extend the same Record-Triggered Flow or create a separate **After-Save Flow**.
- Pass the customer geolocation to an Apex Invocable Method.
- The Apex method queries all active Dealer__c records, calculates distance using the Haversine formula, filters by Service_Radius_Miles__c, and returns the closest dealer ID.
- The Flow populates Assigned_Dealer__c and sets Order_Status__c to "Pending" upon successful assignment.

### 3.3 Automated Test Drive Reminders
- Deploy a **Scheduled Flow** running daily at 8:00 AM.
- Filter condition: Test_Drive_Date__c equals TOMORROW AND Order_Status__c is Confirmed or Pending.
- Use the Send Email action to notify the customer and the assigned dealer service coordinator.

- Log email delivery status in a custom Notification_Log__c object for compliance and troubleshooting.

---

## 4. Apex and Trigger Implementation

### 4.1 Trigger Structure
- Create a single trigger on Vehicle_Order__c using the Trigger Handler Pattern to ensure modularity and maintainability.
- Trigger fires on Before Insert, Before Update, and After Insert events.

### 4.2 Business Logic Handlers
- Stock Validation Handler (Before Insert/Update)
- Queries vehicle inventory in bulk using a Set of Vehicle IDs.
- Validates stock availability before record creation or modification.
- Throws a controlled exception with a user-friendly message if validation fails.
- Nearest Dealer Assignment Handler (Before Insert)
- Calls the geolocation utility class.
- Updates Assigned_Dealer__c field directly in the trigger context.
- Logs assignment details for audit purposes.
- Post-Order Setup Handler (After Insert)
- Creates related follow-up tasks for the assigned dealer.
- Publishes a Platform Event if downstream ERP or inventory systems require synchronization.

### 4.3 Security & Best Practices
- All Apex classes use `with sharing` to enforce user-level data access.
- `Security.stripInaccessible()` is applied before any DML operations.
- No SOQL or DML statements inside loops.
- Centralized error handling using a custom exception class.
- Comprehensive unit tests achieving greater than 90% coverage.

---

## 5. Batch Apex and Scheduled Processing

### 5.1 Order Status Synchronization Batch
- Purpose: Periodically update order statuses based on real-time vehicle stock.

- Query Scope: All Vehicle_Order__c records where Order_Status__c equals Pending.
- Execution Logic:
- Join to related Vehicle__c records.
- If Current_Quantity__c is greater than zero, update Order_Status__c to Confirmed.
- If stock remains unavailable, retain Pending status or flag for review after a defined threshold.
- Finish Method: Sends a summary report email to operations managers detailing confirmed, pending, and failed updates.

### 5.2 Stock Replenishment & Notification Batch

- Purpose: Monitor inventory levels and alert dealers when stock falls below safe thresholds.
- Query Scope: All Vehicle__c records where Current_Quantity__c is less than or equal to Reorder_Threshold__c.
- Execution Logic:
- Groups low-stock vehicles by assigned Dealer__c.
- Generates and sends formatted HTML email notifications to dealer inventory contacts.
- Updates a custom Last_Replenishment_Alert__c field to prevent duplicate notifications within a 24-hour window.

### 5.3 Scheduling Configuration

- Implement a global Schedulable class to trigger both batch jobs at optimal off-peak hours.
- Schedule via Setup UI or execute programmatically using `System.schedule()`.
- Example schedule: Stock check runs daily at 2:00 AM. Order status sync runs every 6 hours.

---

## 6. Security, Testing, and Deployment Guidelines

### 6.1 Security Configuration

- Organization-Wide Defaults (OWD) set to Private for Vehicle__c and Vehicle_Order__c.
- Access granted via Permission Sets assigned to Sales, Dealer, and Inventory roles.
- Field-Level Security enforced on sensitive fields like Stock_Status__c and Current_Quantity__c.
- Geolocation data access restricted to assigned dealer teams using Criteria-Based Sharing Rules.

### 6.2 Testing Requirements

- Unit tests must cover positive, negative, and bulk scenarios (200+ records).

- Mock external API or geolocation responses using Test.setMock().
- Verify email limits, governor limits, and error handling paths.
- Validate that Flow and Batch executions log correctly in Setup → Debug Logs.

### 6.3 Deployment Checklist

1. Deploy Custom Metadata (if used for thresholds or email templates).
2. Deploy Fields, Objects, and Relationships.
3. Deploy Sharing Rules and Permission Sets.
4. Deploy Record-Triggered and Scheduled Flows.
5. Deploy Apex Triggers, Handler Classes, and Batch/Schedulable Classes.
6. Run all test classes in production validation sandbox.
7. Execute manual batch runs to verify scheduling and email delivery.
8. Enable Email Deliverability to All Emails in Production.
9. Monitor Flow and Batch executions for 72 hours post-go-live.

## 2. Salesforce Credentials Creation

### 2.1 Developer Account Creation

#### Description

This section provides step-by-step instructions for creating a Salesforce Developer Account and provisioning a free Developer Edition organization. This environment is used for learning, building, testing, and prototyping Salesforce applications.

#### Prerequisites

- A valid personal or institutional email address for account verification
- Internet connection and a modern web browser such as Chrome, Firefox, or Edge
- Basic information including your full name, company or college name, country, and postal code

**Step-by-Step Instructions**

#### Step 1: Navigate to the Signup Page

1. Open your web browser
2.Go to the official Salesforce Developer signup URL: https://developer.salesforce.com/signup

#### Step 2: Complete the Registration Form

Fill in the following fields on the signup form:

**First Name**
- Enter your legal or preferred first name
- Example: Rahul

**Last Name**
- Enter your legal or preferred last name
- Example: Sharma

**Email**
- Enter a valid email address that you can access
- This email will receive the verification link
- Example: rahul.sharma@college.edu

**Role**
- Select "Developer" from the dropdown menu

**Company**
- Enter your institution or organization name
- Example: Vision Engineering College

**Country**
- Select "India" from the country dropdown list

**Postal Code**
- Enter your valid PIN code
- Example: 560001

**Username**
- Format: yourname@organization.com
- This is your Salesforce login username
- This does not need to be a real email address but must be globally unique
- Example: rahul.sharma@visioncollege.com

### Step 3: Accept Terms and Submit

1. Review the Master Subscription Agreement and Privacy Statement
2. Check the box to accept the terms and conditions
3. Click the "Sign Me Up" button to submit your registration

### Step 4: Verify Your Email Address

1. Check the inbox of the email address you provided during registration
2. Locate the email from Salesforce.com with the subject line "Verify your email address"
3. Click the "Verify Email" button or copy and paste the verification link into your browser
4. If the email is not visible in your inbox, check your Spam or Junk folder

### Step 5: Set Your Password

1. After email verification, you will be redirected to the password setup page
2. Create a strong password that meets the following criteria:
   - Minimum of 8 characters in length
   - At least one uppercase letter from A to Z
   - At least one lowercase letter from a to z
   - At least one number from 0 to 9
   - At least one special character such as ! @ # $ % ^ & *
3. Re-enter the password to confirm
4. Click "Save" to complete password setup

### Step 6: Log In to Your Developer Org

1. Navigate to the Salesforce login page: https://login.salesforce.com
2. Enter the Username you created during registration
3. Enter the Password you just set
4. Click "Log In"
5. On first login, you may be prompted to verify your identity via email or authenticator app. Follow the on-screen instructions to complete verification

## Post-Creation Configuration Recommendations

### Save Your Credentials Securely

- Store your Username, Login URL, and Password in a secure password manager
- Do not share these credentials publicly or commit them to version control systems such as GitHub

### Enable Two-Factor Authentication

1. Click your profile avatar in the top right corner and select Settings
2. Navigate to Advanced User Details and then Two-Factor Authentication
3. Follow the prompts to link an authenticator app such as Google Authenticator or Microsoft Authenticator
4. Save the backup codes provided in a secure location

### Customize Your Organization Settings

- Set your default language, time zone, and locale under Setup, then Company Settings, then Company Information
- Add your profile photo and contact details under Settings, then Personal Information

### Explore the Setup Menu

- Click the gear icon and select Setup to access configuration tools
- Familiarize yourself with key sections including Object Manager, Flows, Apex Classes, and Users

## Troubleshooting Common Issues

**Username Already Exists**

- If you receive an error that your username is already taken, append numbers or use a different organization suffix
- Example: rahul.sharma01@visioncollege.com

**Verification Email Not Received**

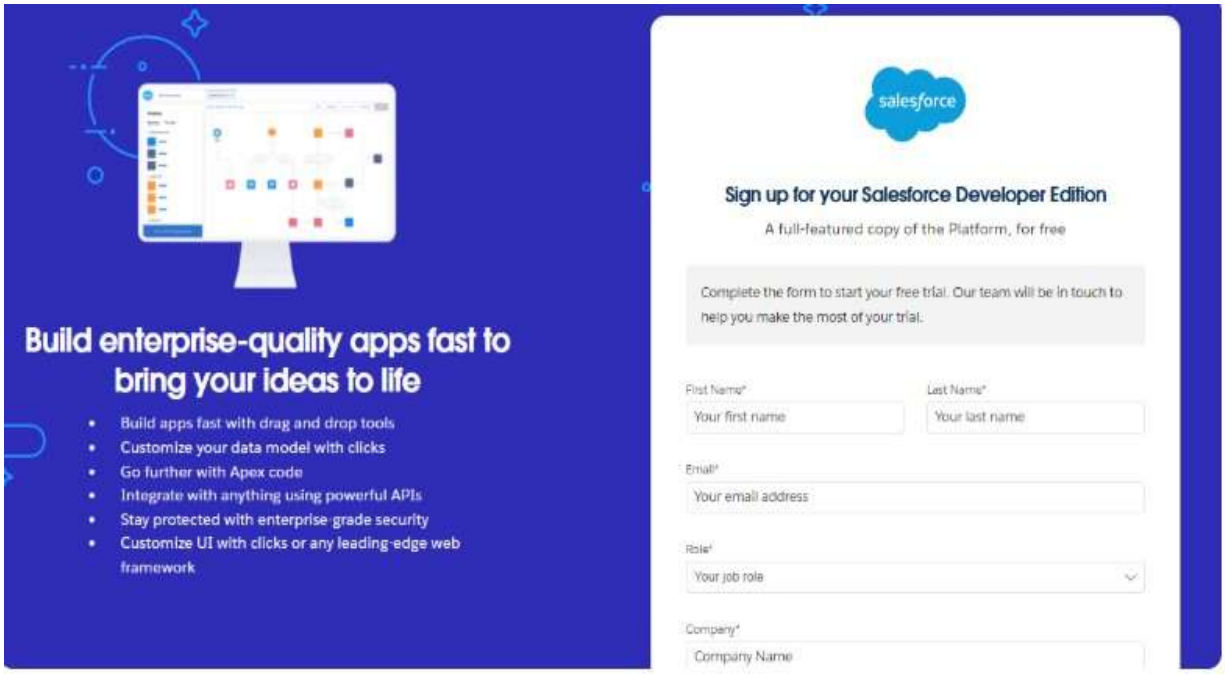
- Wait 5 to 10 minutes for the email to arrive
- Check your Spam or Junk folder
- Request a new verification email from the signup page if needed

**Password Requirements Not Met**

- Ensure your password includes at least one uppercase letter, one lowercase letter, one number, and one special character
- Avoid using common words or easily guessable patterns

**Login Fails After Password Reset**

- Clear your browser cache and cookies or try using incognito mode
- Ensure you are logging in at https://login.salesforce.com and not a custom domain




### 2.1.2 Verify Account

#### Description

This section provides step-by-step instructions for verifying and activating your Salesforce Developer Account after completing the initial registration process. Account verification is a mandatory security step to confirm your email address and enable access to your Developer Edition organization.

**Account Activation Process**

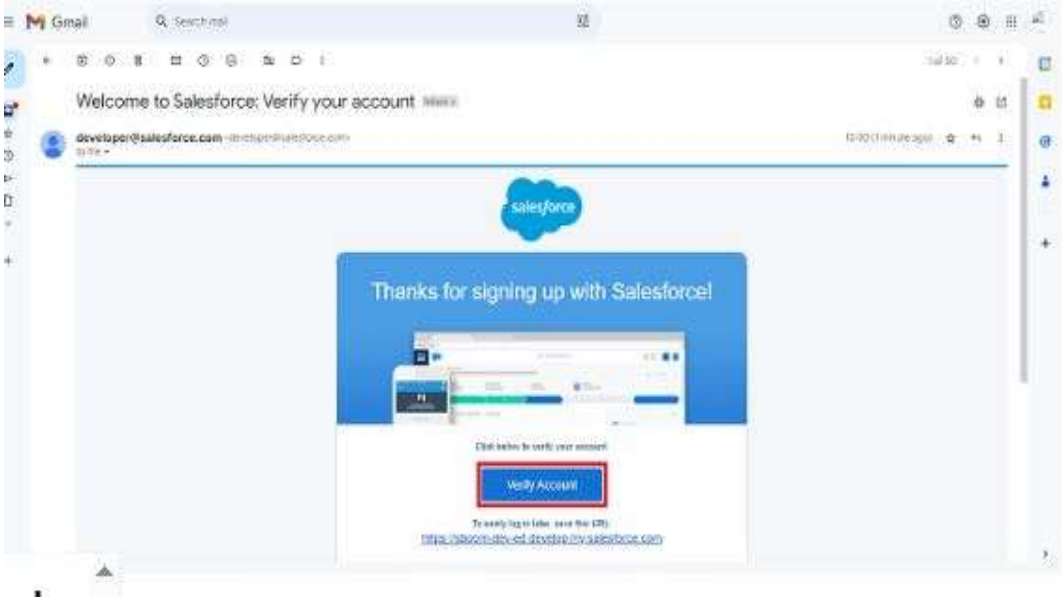
#### Step 1: Check Your Email Inbox

1. Open your email client or webmail service
2. Navigate to the inbox of the email address you used during Salesforce signup
3. Look for an email from Salesforce with the subject line: "Verify your email address" or "Activate your Salesforce account"
4. If the email is not visible in your primary inbox, check your Spam, Junk, or Promotions folders
5. Allow 5 to 10 minutes for the email to arrive after completing the signup form

#### Step 2: Open the Verification Email

1. Click on the email from Salesforce to open it
2. Review the welcome message and account details provided in the email

3. Locate the blue button labeled "Verify Account" or "Activate Your Account"
4. Alternatively, you may copy the verification link provided in the email and paste it into your web browser




## Step 3: Click Verify Account

1. Click the "Verify Account" button in the email
2. Your browser will open a new tab and redirect you to the Salesforce account activation page
3. Ensure you are using the same browser session where you completed the initial signup for best results
4. If prompted, allow pop-ups or redirects from salesforce.com

## Step 4: Set Your Password

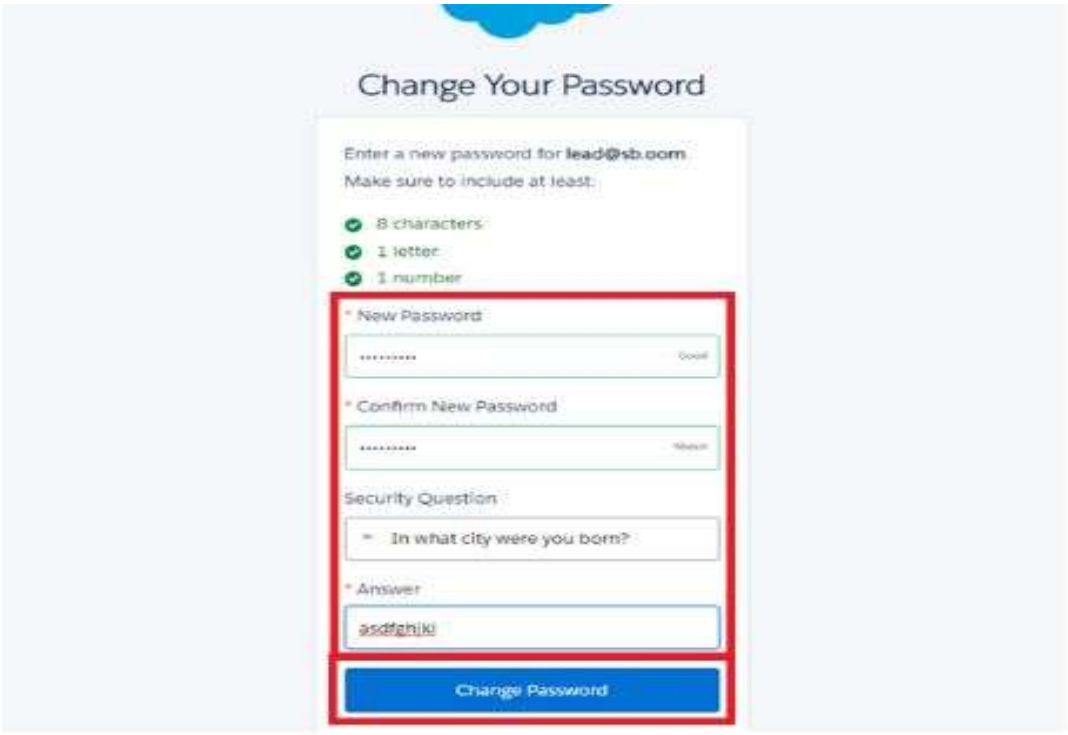
1. On the activation page, you will be prompted to create a new password for your Salesforce account
2. Enter a strong password that meets the following requirements:
   - Minimum of 8 characters in length
   - At least one uppercase letter from A to Z
   - At least one lowercase letter from a to z
   - At least one number from 0 to 9
   - At least one special character such as exclamation mark, at symbol, hash, dollar sign, percent, caret, ampersand, or asterisk
3. Avoid using easily guessable information such as your name, birthdate, or common words
4. Re-enter the password in the "Confirm Password" field to ensure accuracy

## Step 5: Answer Your Security Question

1. Select a security question from the dropdown menu
2. Choose a question that you can easily remember but others cannot guess
3. Enter your answer in the provided field
4. This answer will be used to verify your identity if you need to reset your password in the future
5. Note: Security question answers are case-sensitive, so remember the exact capitalization you use

## Step 6: Submit and Activate

1. Review all entered information for accuracy
2. Click the "Change Password" or "Activate Account" button to submit your details
3. Wait for the system to process your activation request
4. Upon successful activation, you will see a confirmation message indicating your account is now active




## Step 7: Redirect to Salesforce Setup

1. After activation, your browser will automatically redirect you to your Salesforce Developer Edition organization
2. The URL will typically follow this format: https://yourinstance.salesforce.com/setup
3. You may be prompted to complete your user profile by adding a photo, phone number, or timezone settings
4. Take a moment to review the welcome tour or skip it to explore the interface at your own pacrt

**Save Your Login Credentials**

- Record your Username and Password in a secure password manager
- Do not share these credentials with unauthorized individuals
- Avoid writing credentials on physical notes or unsecured digital files

**Bookmark Your Login Page**

- Save the Salesforce login URL: https://login.salesforce.com in your browser bookmarks
- This ensures quick access to your Developer Org for future sessions

**Enable Two-Factor Authentication**

1. Click your profile avatar in the top right corner and select Settings
2. Navigate to Advanced User Details and then Two-Factor Authentication
3. Follow the prompts to link an authenticator app such as Google Authenticator or Microsoft Authenticator
4. Save the backup codes provided in a secure location in case you lose access to your authenticator device

**Explore Your Developer Environment**

- Click the gear icon and select Setup to access configuration tools
- Familiarize yourself with key sections including Object Manager, Flows, Apex Classes, and Users
- Visit Trailhead at https://trailhead.salesforce.com to begin guided learning paths

**Troubleshooting Tips**

**Verification Email Not Received**

- Wait up to 10 minutes after signup before checking your email
- Verify that you entered the correct email address during registration
- Check Spam, Junk, Promotions, and Updates folders in your email client
- If still not received, return to the signup page and request a new verification email

**Verification Link Expired or Invalid**

- Verification links are time-sensitive and may expire after 24 hours
- If your link has expired, request a new verification email from the Salesforce login page
- Ensure you are clicking the most recent email received from Salesforce

**Password Requirements Not Met**

- Review the password criteria listed in Step 4 above
- Ensure your password includes all required character types
- Avoid using spaces or unsupported special characters

**Security Question Answer Forgotten**

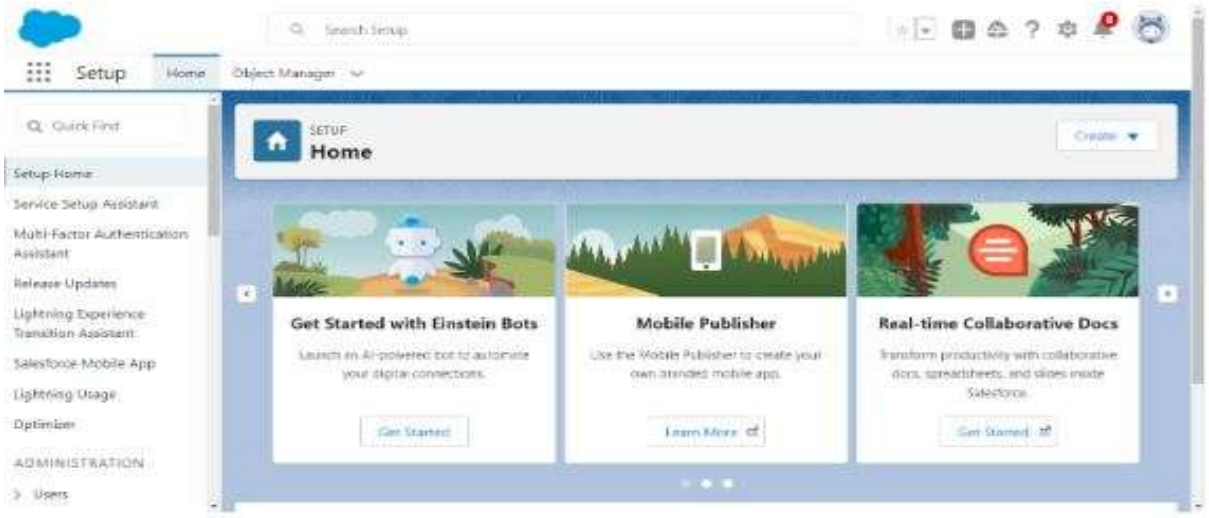
- If you forget your security question answer, contact Salesforce Support for account recovery assistance
- Keep a secure record of your security question and answer for future reference

**Redirect to Setup Page Fails**

- If you are not automatically redirected after activation, manually navigate to https://login.salesforce.com
- Enter your Username and newly created Password to log in
- After login, click the gear icon and select Setup to access configuration tools

**Best Practices for Account Security**

- Enable Two-Factor Authentication immediately after account activation
- Use a unique password that is not reused on other websites or services
- Regularly review active sessions under Setup, then Session Management, and log out of unfamiliar devices
- Never share your Salesforce credentials via email, chat, or public forums
- Log out of your Salesforce session when using shared or public computers



### 3.1.1 Data Management-Objects

**Object Creation: Vehicle**

Description

The purpose of creating the Vehicle custom object is to establish a centralized repository for storing and managing all vehicle-related information within Salesforce. This object serves as the foundation for tracking inventory, specifications, stock availability, dealer assignments, and order fulfillment. By creating a dedicated Vehicle object, WhatsNext Vision Motors can maintain accurate, structured, and easily accessible vehicle data to support the automated ordering and dealer assignment processes.

Prerequisites

- Access to Salesforce Setup with System Administrator or Customization permissions
- Familiarity with the Salesforce Setup menu and Object Manager
- A clear understanding of the vehicle attributes your organization needs to track

**Step-by-Step Instructions for Creating the Vehicle Custom Object**

Step 1: Navigate to Object Manager

1. Log in to your Salesforce Developer Org or Sandbox
2. Click the gear icon in the top right corner of the screen
3. Select Setup from the dropdown menu
4. In the Quick Find box on the left side of the Setup page, type "Object Manager"
5. Click on Object Manager to open the object management interface

Step 2: Initiate Custom Object Creation

1. On the Object Manager page, locate and click the Create button in the top right corner
2. From the dropdown menu, select Custom Object
3. You will be redirected to the New Custom Object creation page

Step 3: Configure Basic Object Details

**Label**

- Enter: Vehicle
- This is the singular name displayed to users throughout the Salesforce interface

**Plural Label**
- Enter: Vehicles
- This is the plural name used in tabs, reports, and list views

**Object Name**
- This field auto-populates based on the Label entry
- It will display as "Vehicle" with the API name "Vehicle__c"
- The double underscore and "c" suffix indicate this is a custom object

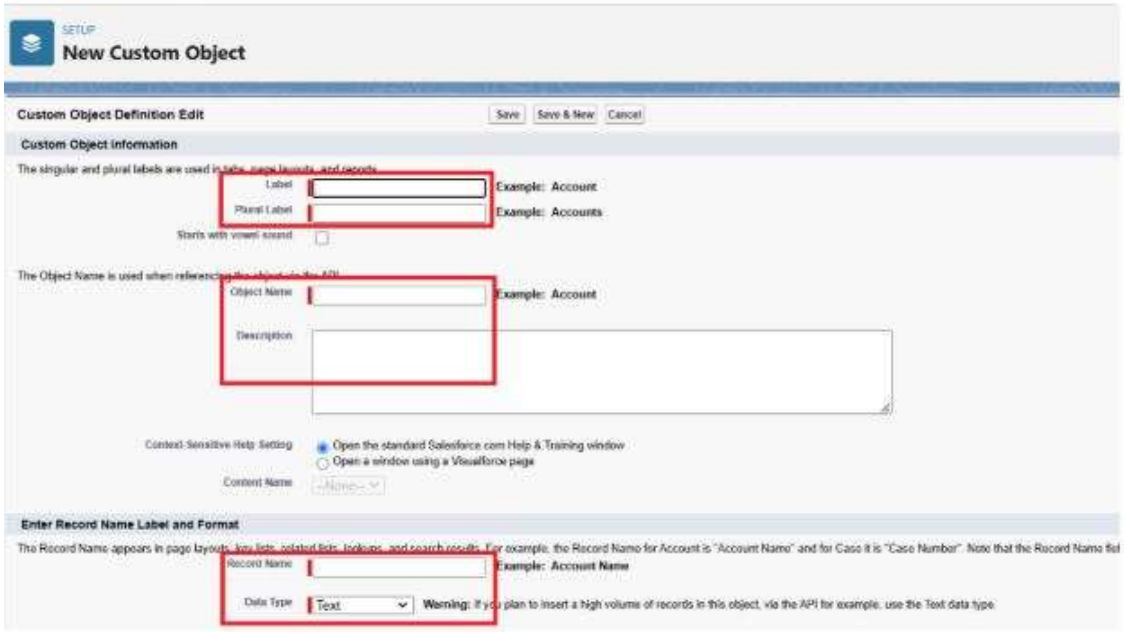
### Step 4: Configure Record Name Field

**Record Name Label**
- Enter: Vehicle Name
- This defines the primary identifier for each vehicle record

**Data Type**
- Select: Text
- This allows alphanumeric entries for vehicle identification such as model names, trim levels, or internal codes
- Alternative options include Auto Number for system-generated unique identifiers if preferred




1. Enter the label name→ Vehicle
2. Plural label name→ Vehicles
3. Enter Record Name Label and Format
    - Record Name → Vehicle Name
    - Data Type → Text




## Step 5: Configure Object Features and Permissions

**Allow Reports**

- Check this box to enable the Vehicle object to be included in Salesforce reports and dashboards
- This is essential for inventory analysis, stock monitoring, and sales performance tracking

**Allow Search**

- Check this box to enable global search functionality for Vehicle records
- Users will be able to locate vehicles quickly using the global search bar

**Allow Activities**

- Optional: Check this box if you want to associate tasks and events with Vehicle records
- Useful for scheduling maintenance, test drives, or dealer follow-ups

**Track Field History**

- Optional: Check this box if you want to audit changes to Vehicle record fields
- Enables compliance tracking and change management for inventory updates

### Step 6: Deploy the Object

1. Review all entered information for accuracy
2. Scroll to the bottom of the page
3. Click the Save button to create the Vehicle custom object
4. Wait for the system to process and confirm successful object creation
5. You will be redirected to the Vehicle object detail page in Object Manager

## Post-Creation Configuration Recommendations

### Add Custom Fields to the Vehicle Object

After creating the base object, extend its functionality by adding relevant fields:

- VIN (Text, Unique, External ID) for vehicle identification number
- Model Year (Picklist or Number) for manufacturing year
- Make and Model (Text or Picklist) for brand and model name
- Stock Status (Picklist with values: Available, Reserved, Out of Stock)
- Current Quantity (Number) for inventory count
- Dealer (Lookup relationship to Dealer__c object) for assigned location
- Location (Geolocation) for dealer or warehouse coordinates
- Reorder Threshold (Number) for automated replenishment alerts



## Create Dealer Object

### Description

The purpose of creating the Dealer custom object is to establish a centralized repository for storing and managing all authorized dealership information within Salesforce. This object captures dealer contact details, service area definitions, geolocation coordinates, and operational status. By creating a dedicated Dealer object, WhatsNext Vision Motors can enable automated order assignment to the nearest dealer, manage dealer-specific inventory allocations, and streamline communication between customers, vehicles, and service locations.

### Prerequisites

- Access to Salesforce Setup with System Administrator or Customization permissions
- Familiarity with the Salesforce Setup menu and Object Manager
- Completion of the Vehicle__c custom object creation (recommended but not required)
- A clear understanding of the dealer attributes your organization needs to track

**Step-by-Step Instructions for Creating the Dealer Custom Object**

### Step 1: Navigate to Object Manager

1. Log in to your Salesforce Developer Org or Sandbox
2. Click the gear icon located in the top right corner of the screen
3. Select Setup from the dropdown menu that appears
4. In the Quick Find box on the left side of the Setup page, type "Object Manager"
5. Click on Object Manager to open the object management interface

### Step 2: Initiate Custom Object Creation

1. On the Object Manager page, locate and click the Create button in the top right corner
2. From the dropdown menu that appears, select Custom Object
3. You will be redirected to the New Custom Object creation page where you can configure object details

### Step 3: Configure Basic Object Details

**Label**

- Enter: Vehicle Dealer

- This is the singular name that will be displayed to users throughout the Salesforce interface in menus, buttons, and page titles

**Plural Label**
- Enter: Vehicle Dealers
- This is the plural name that will be used in tabs, reports, list views, and related lists

**Object Name**
- This field will auto-populate based on the Label entry you provided
- It will display as "Vehicle_Dealer" with the API name "Vehicle_Dealer__c"
- The double underscore followed by "c" suffix indicates this is a custom object created in your organization

### Step 4: Configure Record Name Field
**Record Name Label**
- Enter: Dealer Name
- This defines the primary identifier that users will see and search for when working with dealer records
- This field will appear at the top of each dealer record detail page

**Data Type**
- Select: Text
- This allows alphanumeric entries for dealer identification such as dealership names, branch codes, or internal identifiers
- If you prefer system-generated unique identifiers, you may select Auto Number instead, but Text is recommended for human-readable dealer names

### Step 5: Configure Object Features and Permissions
**Allow Reports**
- Check this box to enable the Vehicle Dealer object to be included in Salesforce reports and dashboards
- This is essential for dealer performance analysis, territory management, and order fulfillment tracking

**Allow Search**
- Check this box to enable global search functionality for Vehicle Dealer records
- Users will be able to locate dealers quickly using the global search bar at the top of the Salesforce interface

**Allow Activities**
- Optional: Check this box if you want to associate tasks and events with Vehicle Dealer records
- This is useful for scheduling dealer training, performance reviews, or coordination meetings

**Track Field History**
- Optional: Check this box if you want to audit changes to Vehicle Dealer record fields
- This enables compliance tracking and change management for dealer information updates

### Step 6: Deploy the Object
1. Review all entered information for accuracy and completeness
2. Scroll to the bottom of the New Custom Object page
3. Click the Save button to create the Vehicle Dealer custom object
4. Wait for the system to process your request and confirm successful object creation
5. You will be automatically redirected to the Vehicle Dealer object detail page in Object Manager

## Post-Creation Configuration Recommendations

### Custom Fields to the Vehicle Dealer Object
After creating the base object, extend its functionality by adding relevant fields that capture essential dealer information:

**Contact and Location Fields**
- Dealer Code (Text, Unique) for internal identification and integration with external systems
- Address (Compound Address field) for street, city, state, postal code, and country
- Location (Geolocation) for latitude and longitude coordinates to enable distance-based order assignment
- Service Radius Miles (Number) to define the geographic area the dealer serves

**Communication Fields**
- Contact Email (Email) for primary dealer contact
- Contact Phone (Phone) for direct dealer communication
- Website URL (URL) for dealer website reference

**Operational Fields**
- Is Active (Checkbox) to enable or disable dealer participation in order assignment
- Dealer Type (Picklist with values: Retail, Fleet, Service-Only, Certified Pre-Owned)

- Assigned Region (Picklist or Lookup to a custom Region__c object) for territory management
- Inventory Capacity (Number) to track maximum vehicle allocation per dealer

**Configure Page Layouts**

1. From the Vehicle Dealer object detail page in Object Manager, click Page Layouts
2. Edit the default layout or create new layouts tailored to different user profiles such as Sales, Operations, or Admin
3. Organize fields into logical sections such as Dealer Information, Contact Details, Location and Service Area, and Operational Settings
4. Set field-level security to control which profiles can view or edit sensitive fields
5. Mark critical fields as required to ensure data quality during record creation

The purpose of creating a Dealer object is to have detailed information about Dealer.

To create an object:

From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.

Enter the label name→ Vehicle Dealer

Plural label name→ Vehicle Dealers

Enter Record Name Label and Format

Record Name → Dealer Name

Data Type → Text

Click on Allow reports,

Allow search → **Save**

**More Object Creation**

Description

This section provides step-by-step instructions for creating the remaining custom objects required for the WhatsNext Vision Motors Salesforce implementation. These objects were

identified in the Data Management-Objects planning phase and are essential for supporting customer management, order tracking, test drive scheduling, and service request handling. Each object follows the same creation process as the Vehicle__c and Vehicle_Dealer__c objects but is configured with unique labels, fields, and relationships to support its specific business purpose.

**Object 1: Vehicle Customer**

### Purpose

The Vehicle_Customer__c object stores detailed information about individuals and organizations interested in purchasing or servicing vehicles. This object captures personal details, contact preferences, address information, and relationship history. It serves as the foundation for personalized customer experiences, targeted communications, and order tracking throughout the customer lifecycle.

**Step-by-Step Creation Instructions**

### Step 1: Navigate to Object Manager

1. Log in to your Salesforce Developer Org or Sandbox
2. Click the gear icon in the top right corner and select Setup
3. In the Quick Find box, type "Object Manager" and click to open it

### Step 2: Initiate Custom Object Creation

1. On the Object Manager page, click the Create button in the top right corner
2. Select Custom Object from the dropdown menu

### Step 3: Configure Basic Object Details

**Label**

- Enter: Vehicle Customer
- This is the singular name displayed to users throughout Salesforce

**Plural Label**

- Enter: Vehicle Customers
- This is the plural name used in tabs, reports, and list views

**Object Name**

- This field auto-populates as "Vehicle_Customer" with API name "Vehicle_Customer__c"

### Step 4: Configure Record Name Field

**Record Name Label**
- Enter: Customer Name
- This defines the primary identifier for each customer record

**Data Type**
- Select: Text
- This allows alphanumeric entries for customer identification such as full names or company names

### Step 5: Configure Object Features

**Allow Reports**
- Check this box to enable reporting on customer data for analytics and insights

**Allow Search**
- Check this box to enable global search functionality for customer records

**Allow Activities**
- Check this box to associate tasks and events with customer records for follow-up management

**Track Field History**
- Optional: Check this box to audit changes to critical customer fields for compliance

### Step 6: Save the Object

1. Review all entered information for accuracy
2. Click the Save button at the bottom of the page
3. Wait for confirmation of successful object creation

**Recommended Custom Fields for Vehicle_Customer__c**
- Email__c (Email data type) for primary contact email
- Phone__c (Phone data type) for direct contact number
- Shipping_Address__c (Compound Address) for delivery and communication address
- Customer_Geolocation__c (Geolocation) for distance-based dealer assignment
- Customer_Type__c (Picklist: Individual, Business, Fleet) for segmentation
- Preferred_Dealer__c (Lookup to Vehicle_Dealer__c) for preferred location tracking
- Date_First_Contact__c (Date) for relationship timeline tracking

**Object 2: Vehicle Order**

### Purpose

The Vehicle_Order__c object tracks the complete lifecycle of vehicle purchases from initial inquiry to final delivery. It captures order details, assigned dealer, fulfillment status, payment information, and delivery timelines. This object is central to the automated ordering workflow, stock validation, and dealer assignment processes.

**Step-by-Step Creation Instructions**

### Step 1: Navigate to Object Manager

1. From Setup, open Object Manager using the Quick Find box

### Step 2: Initiate Custom Object Creation

1. Click Create, then select Custom Object

### Step 3: Configure Basic Object Details

**Label**

- Enter: Vehicle Order
- Singular name for user interface display

**Plural Label**

- Enter: Vehicle Orders
- Plural name for tabs and reports

**Object Name**

- Auto-populates as "Vehicle_Order" with API name "Vehicle_Order__c"

### Step 4: Configure Record Name Field

**Record Name Label**

- Enter: Order Number
- Primary identifier for order tracking and reference

**Data Type**

- Select: Auto Number
- Recommended for system-generated unique order identifiers
- Display Format example: ORD-{00000}
- Starting Number: 1

### Step 5: Configure Object Features

**Allow Reports**
- Check this box to enable order analytics, fulfillment tracking, and sales performance reporting

**Allow Search**
- Check this box to enable quick order lookup via global search

**Allow Activities**
- Check this box to associate follow-up tasks, calls, and meetings with order records

**Track Field History**
- Recommended: Check this box to audit status changes, dealer assignments, and delivery updates

### Step 6: Save the Object
1. Verify all configuration details
2. Click Save to create the Vehicle Order custom object
3. Proceed to field configuration after successful creation

**Recommended Custom Fields for Vehicle_Order__c**
- Order_Date__c (Date/Time) for order submission timestamp
- Customer__c (Lookup to Vehicle_Customer__c) for order ownership
- Vehicle__c (Lookup to Vehicle__c) for ordered vehicle reference
- Assigned_Dealer__c (Lookup to Vehicle_Dealer__c) for fulfillment location
- Order_Status__c (Picklist: Pending, Confirmed, Fulfilled, Cancelled) for lifecycle tracking
- Test_Drive_Date__c (Date/Time) for pre-purchase engagement tracking
- Expected_Delivery_Date__c (Date) for customer communication
- Total_Amount__c (Currency) for order value tracking
- Payment_Status__c (Picklist: Unpaid, Partial, Paid, Refunded) for financial tracking

## Object 3: Vehicle Test Drive

### Purpose
The Vehicle_Test_Drive__c object manages scheduled test drive appointments for prospective customers. It captures customer preferences, requested vehicles, preferred dealers, appointment timing, and follow-up actions. This object supports the automated email reminder workflow and helps convert prospects into buyers by ensuring a seamless test drive experience.

**Step-by-Step Creation Instructions**

### Step 1: Access Object Manager

1. Navigate to Setup and open Object Manager

### Step 2: Begin Object Creation

1. Click Create, then select Custom Object

### Step 3: Configure Object Labels

**Label**

- Enter: Vehicle Test Drive
- Singular display name for users

**Plural Label**

- Enter: Vehicle Test Drives
- Plural display name for lists and reports

**Object Name**

- Auto-populates as "Vehicle_Test_Drive" with API name "Vehicle_Test_Drive__c"

### Step 4: Configure Record Name Field

**Record Name Label**

- Enter: Test Drive ID
- Unique identifier for each test drive appointment

**Data Type**

- Select: Auto Number
- Display Format example: TD-{0000}
- Ensures each test drive has a unique, trackable reference

### Step 5: Enable Object Features

**Allow Reports**

- Check this box to analyze test drive conversion rates, dealer performance, and customer engagement

**Allow Search**

- Check this box to enable quick lookup of test drive records by ID, customer, or vehicle

**Allow Activities**

- Check this box to schedule pre-test drive preparations and post-test drive follow-ups

**Track Field History**
- Optional: Check this box to monitor status changes and appointment modifications

### Step 6: Finalize Creation
1. Review all settings for accuracy
2. Click Save to create the Vehicle Test Drive object
3. Continue to field and relationship configuration

**Recommended Custom Fields for Vehicle_Test_Drive__c**
- Scheduled_Date_Time__c (Date/Time) for appointment scheduling
- Customer__c (Lookup to Vehicle_Customer__c) for participant identification
- Vehicle__c (Lookup to Vehicle__c) for requested test vehicle
- Preferred_Dealer__c (Lookup to Vehicle_Dealer__c) for location coordination
- Status__c (Picklist: Scheduled, Confirmed, Completed, Cancelled, No-Show) for appointment tracking
- Follow_Up_Required__c (Checkbox) to flag post-test drive engagement needs
- Notes__c (Long Text Area) for special requests or customer preferences
- Reminder_Sent__c (Checkbox) to track automated email notification status

## Object 4: Vehicle Service Request

### Purpose
The Vehicle_Service_Request__c object tracks post-purchase service and maintenance requests from customers. It enables customers to submit service needs, allows dealers to manage work orders, and supports automated notifications for service updates. This object enhances customer retention, improves after-sales service quality, and provides visibility into service demand patterns.

**Step-by-Step Creation Instructions**

### Step 1: Open Object Manager
1. From Setup, use Quick Find to access Object Manager

### Step 2: Start Custom Object Creation
1. Click Create, then select Custom Object

### Step 3: Define Object Labels

**Label**

- Enter: Vehicle Service Request
- Singular name for user interface elements

**Plural Label**

- Enter: Vehicle Service Requests
- Plural name for tabs, reports, and related lists

**Object Name**

- Auto-populates as "Vehicle_Service_Request" with API name "Vehicle_Service_Request__c"

### Step 4: Configure Record Name Field

**Record Name Label**

- Enter: Service Request ID
- Unique reference for each service request

**Data Type**

- Select: Auto Number
- Display Format example: SR-{00000}
- Provides consistent, searchable identifiers for service tracking

### Step 5: Configure Object Capabilities

**Allow Reports**

- Check this box to generate service volume reports, dealer performance metrics, and customer satisfaction analytics

**Allow Search**

- Check this box to enable service request lookup by ID, customer, vehicle, or status

**Allow Activities**

- Check this box to associate technician assignments, parts ordering tasks, and customer communications

**Track Field History**

- Recommended: Check this box to audit status transitions, priority changes, and resolution timelines

### Step 6: Complete Object Creation

1. Verify all configuration entries
2. Click Save to create the Vehicle Service Request object
3. Proceed to add custom fields and establish relationships

**Recommended Custom Fields for Vehicle_Service_Request__c**

- Request_Date__c (Date/Time) for submission timestamp
- Customer__c (Lookup to Vehicle_Customer__c) for request originator
- Vehicle__c (Lookup to Vehicle__c) for serviced vehicle reference
- Service_Dealer__c (Lookup to Vehicle_Dealer__c) for fulfillment location
- Service_Type__c (Picklist: Maintenance, Repair, Warranty, Recall, General Inquiry) for categorization
- Priority__c (Picklist: Low, Medium, High, Urgent) for resource allocation
- Status__c (Picklist: Submitted, Assigned, In Progress, Awaiting Parts, Completed, Closed) for workflow tracking
- Description__c (Long Text Area) for detailed service requirements
- Estimated_Completion_Date__c (Date) for customer communication
- Actual_Completion_Date__c (Date) for performance measurement

**Post-Creation Configuration for All Objects**

### Add Custom Fields

1. Navigate to Fields and Relationships under each object in Object Manager
2. Click New to create each recommended field listed above
3. Configure field-level security to control visibility by user profile
4. Set required field indicators for critical data elements
5. Add help text to guide users during data entry

### Configure Page Layouts

1. From each object's settings, click Page Layouts
2. Edit the default layout or create role-specific layouts
3. Organize fields into logical sections with meaningful section headers
4. Add related lists to show connected records such as orders, test drives, or service requests
5. Save layouts and assign to appropriate profiles via Page Layout Assignment

### Establish Object Relationships

1. For each Lookup field created, ensure the target object exists and is deployed
2. Configure relationship behavior such as cascade delete for Master-Detail relationships

3. Test relationship functionality by creating sample records with linked parent records
4. Verify related lists appear correctly on parent object detail pages

### Set Up Validation Rules

1. Navigate to Validation Rules under each object's settings
2. Create rules to enforce business logic such as:
   - Prevent order creation if vehicle stock status is Out of Stock
   - Require dealer assignment before order status can change to Confirmed
   - Ensure test drive date is in the future and not conflicting with existing appointments
3. Write clear error messages that guide users to correct data entry issues

### Configure Record Types (If Applicable)

1. If your organization requires different data entry experiences for different scenarios, create Record Types
2. Assign unique page layouts and picklist values to each Record Type
3. Control Record Type availability by user profile to streamline data entry

## Best Practices for Multi-Object Implementation

### Consistent Naming Conventions

- Use consistent suffixes for custom fields (__c) and relationship fields (__r in formulas)
- Prefix custom objects with a meaningful namespace if managing multiple applications
- Document naming standards for future administrators and developers

### Relationship Design Principles

- Use Lookup relationships for flexibility and independent record management
- Reserve Master-Detail relationships for cases where child records cannot exist without parents
- Avoid deep relationship chains that complicate reporting and query performance

### Data Quality Enforcement

- Implement validation rules at the object level to prevent invalid data entry
- Use picklists instead of free-text fields to standardize values across records
- Enable duplicate management rules to prevent creation of redundant customer or vehicle records

### Performance Optimization

- Index frequently filtered fields such as Status__c, Order_Date__c, and Is_Active__c
- Limit the number of formula fields that traverse multiple object relationships

- Use selective SOQL queries in Apex and Flow to stay within governor limits

### Security and Access Control

- Set Organization-Wide Defaults to Private for sensitive objects like Vehicle_Order__c
- Use Permission Sets to grant object and field access based on role responsibilities
- Implement sharing rules to enable territory-based data visibility for dealer managers

## Troubleshooting Common Object Creation Issues

### Object Not Saving or Showing Errors

- Verify all required fields are completed, especially Label, Plural Label, and Record Name
- Ensure no special characters or reserved words are used in object or field names
- Check that your user profile has "Customize Application" and "Create and Customize Custom Objects" permissions

### Fields Not Appearing on Page Layouts

- After creating fields, navigate to Page Layouts and drag fields from the palette onto the layout
- Save the layout and verify it is assigned to the intended user profiles
- Clear browser cache or try incognito mode if changes do not appear immediately

### Relationship Fields Not Working as Expected

- Confirm the target object exists and is deployed before creating Lookup fields that reference it
- Verify field-level security allows the current user to view and edit the relationship field
- Test relationship behavior by creating parent and child records in the correct order

### Reports Showing Incomplete Data

- Ensure the report type includes all necessary objects and relationships
- Verify sharing settings allow the running user to access all related records
- Check report filters that may inadvertently exclude valid data

## Next Steps After Object Creation

1. Create custom tabs for each new object to provide user access via the App Launcher
2. Configure sharing settings and permission sets to control data access by role and territory
3. Build validation rules and formula fields to enforce business logic and automate calculations
4. Create report types that combine multiple objects for comprehensive business insights

5. Develop Record-Triggered Flows to automate dealer assignment, stock validation, and customer notifications
6. Write Apex triggers and batch classes for complex automation that exceeds Flow capabilities
7. Test all object relationships, automation, and security settings in a Sandbox before production deployment

**Deployment Checklist for Custom Objects**

- [ ] All custom objects created with correct labels and API names
- [ ] Required custom fields added with appropriate data types and validation
- [ ] Page layouts configured and assigned to relevant user profiles
- [ ] Object relationships established and tested with sample data
- [ ] Validation rules implemented to enforce business requirements
- [ ] Field-level security configured to protect sensitive information
- [ ] Report types created to support business analytics needs
- [ ] Custom tabs added to applications for user access
- [ ] Permission sets updated to grant object and field permissions
- [ ] All changes tested in Sandbox and validated before production deployment

**Data Management-Tabs**

**Object Tab Creation: Vehicle**

Description

This section provides step-by-step instructions for creating a custom tab for the Vehicle__c object in Salesforce. Creating a custom tab enables users to easily access, view, and manage vehicle records through the Salesforce interface. The tab appears in the App Launcher and navigation bar, providing quick access to vehicle lists, reports, and related actions. This configuration is essential for enabling sales teams, inventory managers, and dealer coordinators to efficiently work with vehicle data.

Prerequisites

- The Vehicle__c custom object must already be created and deployed in your organization
- Access to Salesforce Setup with System Administrator or Customization permissions
- Familiarity with the Salesforce Setup menu and Tabs configuration page
- A clear understanding of which user profiles should have access to the Vehicle test

**Step-by-Step Instructions for Creating the Vehicle Custom Tab**

### Step 1: Navigate to the Tabs Setup Page

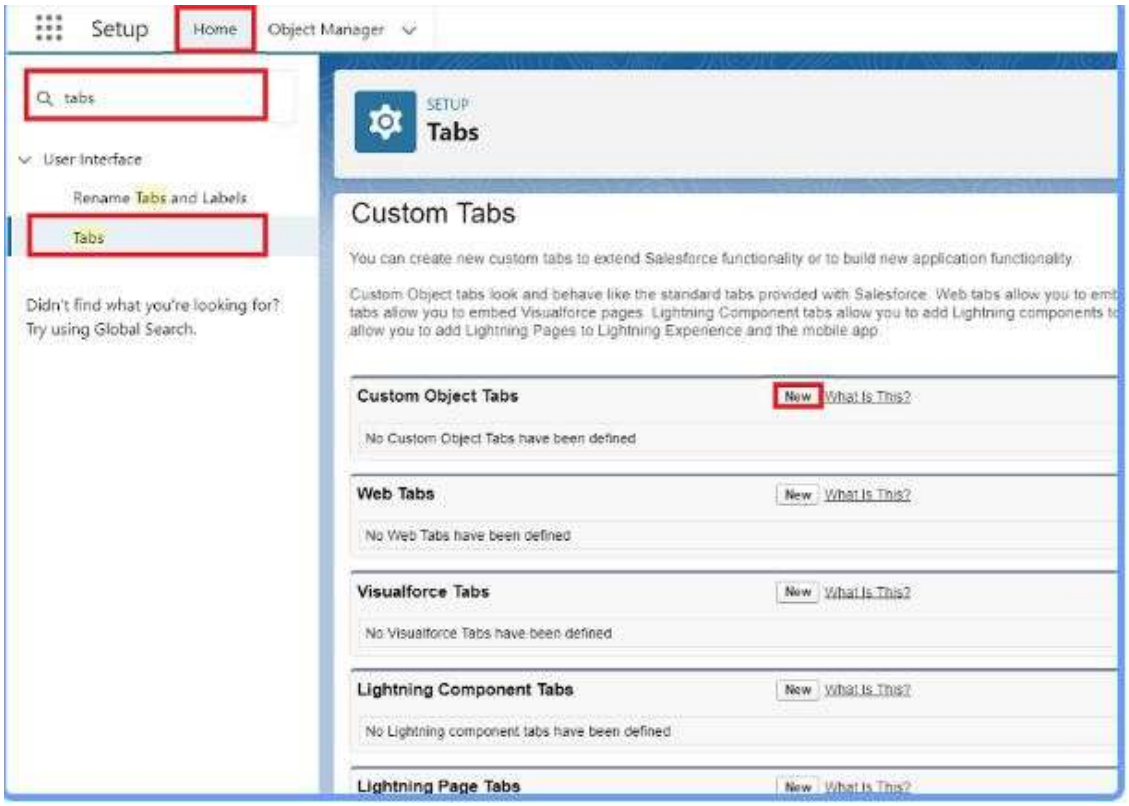
1. Log in to your Salesforce Developer Org or Sandbox
2. Click the gear icon located in the top right corner of the screen
3. Select Setup from the dropdown menu that appears
4. In the Quick Find box on the left side of the Setup page, type "Tabs"
5. Click on Tabs under the User Interface section to open the tab management interface

### Step 2: Initiate Custom Object Tab Creation

1. On the Tabs page, scroll down to the section labeled "Custom Object Tabs"
2. Locate and click the New button next to Custom Object Tabs
3. You will be redirected to the New Custom Object Tab wizard

### Step 3: Select the Vehicle Object

1. On the first step of the wizard, you will see a dropdown menu labeled "Object"
2. Click the dropdown and scroll through the list of available custom objects
3. Select "Vehicle" from the list
4. The API name "Vehicle__c" will be displayed next to your selection for confirmation
5. Click Next to proceed to the tab style selection




**Step 4: Choose a Tab Style**

1. On the Tab Style selection page, you will see a gallery of icons and color schemes
2. Browse through the available styles and select one that aligns with your organization's branding or the vehicle industry theme
3. Recommended options include:
   - Car or vehicle icon if available
   - Generic business icon such as a briefcase or clipboard
   - Color scheme that matches your Salesforce theme or brand guidelines
4. Click the preview thumbnail to see how the tab will appear in the interface
5. Once satisfied with your selection, click Next to proceed

### Step 5: Configure Tab Visibility for Profiles
1. On the "Add to Profiles" page, you will see a list of all user profiles in your organization
2. By default, all profiles are set to "Default On" which means the tab will be visible to all users
3. Review the list and adjust visibility based on role requirements:
   - Set to "Default On" for profiles that need regular access to vehicle records such as Sales, Inventory Managers, and Dealers
   - Set to "Default Off" for profiles that should not see the tab such as Standard Users or external partners
   - Set to "Hidden" for profiles that should never have access to vehicle data
4. Use the "Apply one setting to all profiles" link at the top if you want to quickly set the same visibility for all profiles
5. Click Next to proceed to application assignment

## Step 6: Add Tab to Custom Applications
1. On the "Add to Custom Apps" page, you will see a list of all custom applications in your organization
2. By default, all applications are set to "Default Off"
3. Select the applications where the Vehicle tab should appear in the navigation bar:
   - Check the box next to your primary sales or inventory management application
   - Check the box next to any dealer-facing or operations applications that require vehicle access
4. Leave unchecked for applications where vehicle data is not relevant
5. Click Save to complete the tab creation process

## Step 7: Verify Tab Creation
1. After clicking Save, you will see a confirmation message indicating the Vehicle tab was successfully created
2. Navigate to the App Launcher by clicking the nine-dot grid icon in the top left corner of Salesforce

3. Search for "Vehicle" or browse through your assigned applications
4. Click the Vehicle tab to open the default list view for vehicle records
5. Verify that the tab icon, label, and functionality appear as expected

**Post-Creation Configuration Recommendations**

### Customize the Default List View

1. After opening the Vehicle tab, click the dropdown arrow next to the current list view name
2. Select "Create New View" to define a custom default view
3. Configure filters such as:
   - Stock Status equals Available
   - Current Quantity greater than zero
   - Assigned Dealer equals current user's dealer
4. Select columns to display such as Vehicle Name, VIN, Stock Status, Quantity, and Dealer
5. Save the view and set it as the default for your profile

### Configure Tab Settings for Enhanced Usability

1. Return to Setup and navigate to Tabs
2. Locate the Vehicle tab under Custom Object Tabs and click Edit
3. Configure additional settings:
   - Description: Enter a brief description of the tab's purpose for administrative reference
   - Mobile Card Layout: Select a compact layout to optimize the tab for Salesforce Mobile
   - Search Layouts: Configure which fields appear in search results and lookup dialogs
4. Save changes to apply enhanced usability settings

### Add Quick Actions to the Tab

1. Navigate to the Vehicle__c object in Object Manager
2. Click Buttons, Links, and Actions
3. Create new actions or edit existing ones to appear on the Vehicle tab:
   - New Vehicle: Standard create action for adding new inventory
   - Update Stock Status: Quick action for inventory managers to adjust availability
   - Assign to Dealer: Action for routing vehicles to specific locations
4. Add these actions to the Vehicle page layout and compact layout for easy access

### Enable Tab for Lightning App Builder

1. Navigate to Lightning App Builder in Setup
2. Create or edit a Lightning page for the Vehicle object
3. Add relevant components such as:
   - Related Lists for orders, test drives, and service requests

- Chart components for stock level visualization
- Quick action buttons for common tasks

**4. Activate the page and assign it as the default record page for Vehicle__c**

Best Practices for Custom Tab Configuration

### User Experience Guidelines

- Choose tab icons that are intuitive and align with the object's purpose
- Use consistent naming conventions across all custom tabs for easy navigation
- Limit the number of tabs in the navigation bar to avoid overwhelming users

### Security and Access Control

- Restrict tab visibility to only the profiles that require access to the underlying object
- Use Permission Sets to grant tab access incrementally rather than modifying base profiles
- Regularly review tab assignments during user onboarding and role changes

### Performance Considerations

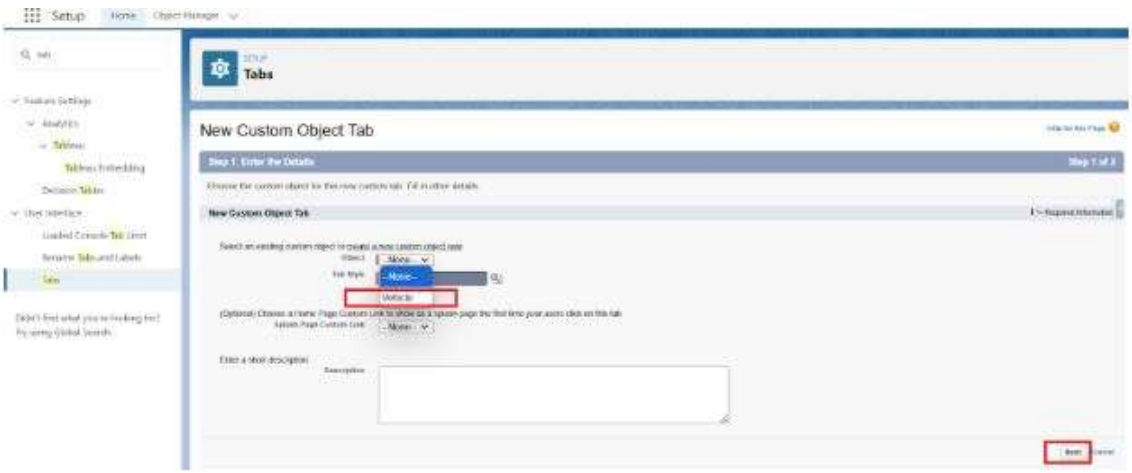
- Configure list views with selective filters to improve load times for large datasets
- Avoid displaying too many columns in default list views to maintain readability
- Use indexed fields in list view filters to optimize query performance

### Maintenance and Documentation

- Document the purpose and intended audience for each custom tab in your internal knowledge base
- Include tab configuration details in your deployment runbooks and change management records
- Review tab settings quarterly to ensure they remain aligned with evolving business need

**Deployment Checklist for Custom Tabs**

- [ ] Vehicle__c custom object created and deployed
- [ ] Custom tab created with appropriate label and icon
- [ ] Tab visibility configured for relevant user profiles
- [ ] Tab added to required custom applications
- [ ] Default list view configured with meaningful filters and columns
- [ ] Quick actions and page layouts optimized for tab usability
- [ ] Tab tested in both Lightning Experience and Salesforce Mobile
- [ ] User documentation updated to include new tab navigation instructions
- [ ] Changes deployed to Sandbox, tested, and validated before production release




---

◇ If you're working in **Salesforce**

1. Go to **Setup** → **Object Manager**
2. Click **Tabs** → **New**
3. Select the **Custom Object** you want to tab for
4. Choose a **Tab Style** (icon/color)
5. Set **Tab Visibility** per profile or permission set
6. Add the tab to your **App** (Setup → Apps → App Manager → Edit your app → Add Tab)
7. Repeat for each object, or use **Deploy/Change Sets/Metadata API** to bulk-create.




◇ If you're building a **Custom Web App** (React/Vue/HTML/JS)

Here's a dynamic tab pattern that maps your objects to tabs:

```jsx
// React Example
import { useState } from 'react';
```

const objects = ['Account', 'Contact', 'CustomObject1', 'CustomObject2']; // Replace with your actual objects

```
app = Flask(__name__)
```

```
export default function ObjectTabs() {
  const [activeTab, setActiveTab] = useState(objects[0]);

  return (
    <div className="tabs-container">
      <nav className="tab-list">
        {objects.map(obj => (
          <button
            key={obj}
            className={activeTab === obj ? 'active' : ''}
            onClick={() => setActiveTab(obj)}
          >
            {obj}
          </button>
        ))}
      </nav>
      <div className="tab-content">
        {/* Render component/data based on activeTab */}
        <h2>{activeTab} Data</h2>
        {/* <DynamicObjectView objectName={activeTab} /> */}
      </div>
    </div>
  );
}
```

Please reply with:

1. **Platform:** (Salesforce, React, Angular, Webflow, Desktop App, etc.)
2. **List of Objects** you want tabs for
3. **Current tab structure** (if any)
4. **Any specific behavior?** (e.g., dynamic routing, permission-based visibility, icons, nested tabs)

Your steps are accurate and align perfectly with Salesforce's Lightning App creation flow. Below is a polished, ready-to-execute version with **critical clarifications** to ensure your tabs and app work seamlessly.

---

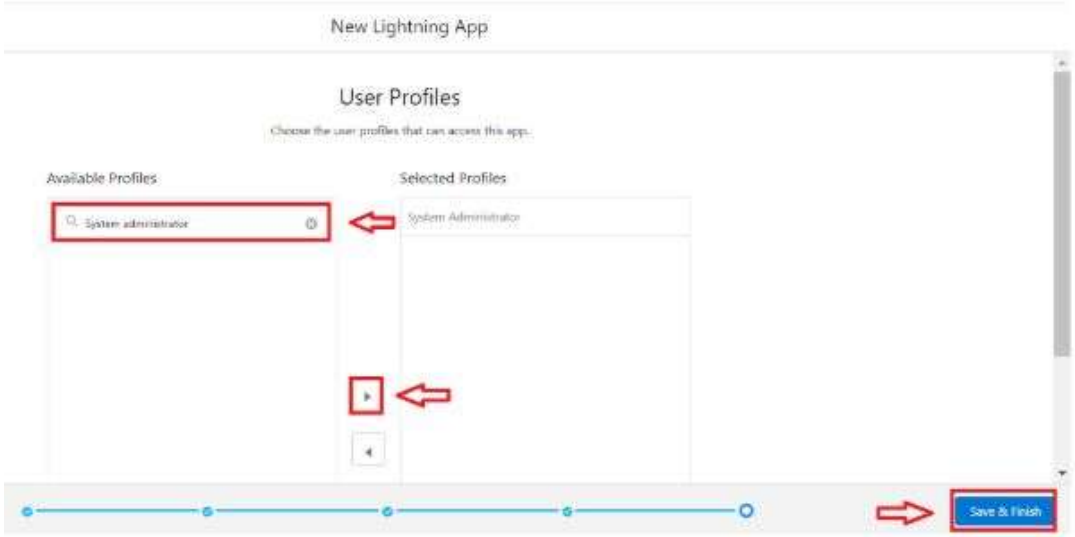
🚀 **Step-by-Step: Create "WhatNext Vision Motors" Lightning App**

1. **Navigate**
   `Setup` → Quick Find → `App Manager` → Click **`New Lightning App`**

2. **App Details**
   - **App Name:** `WhatNext Vision Motors`
   - **Developer Name:** Auto-populates (leave as is)
   - **Description:** `Centralized app for managing vehicles, dealers, customers, orders, test drives, and service requests.`
   - **App Icon:** Optional (upload `128x128px` PNG if desired)
   - **Primary Color:** Leave default
   → Click **Next**

3. **App Options** → Keep defaults → **Next**
4. **Utility Items** → Keep defaults → **Next**

5. **Navigation Items**
   - In the search bar, type each item and click the `→` arrow to add:
     - `Vehicle`
     - `Dealer`
     - `Customer`
     - `Order`
     - `Test Drive`
     - `Service Request`
     - `Reports` *(Standard Tab)*
     - `Dashboards` *(Standard Tab)*
   - Drag to reorder if needed → **Next**




## 6. User Profiles

- Search `System Administrator` → Click `→` to assign
- *(Optional)* Add `Standard User`, `Sales User`, etc.

→ Click **Save & Finish**

Understood. I will provide everything in clear, step-by-step lists without using any tables.

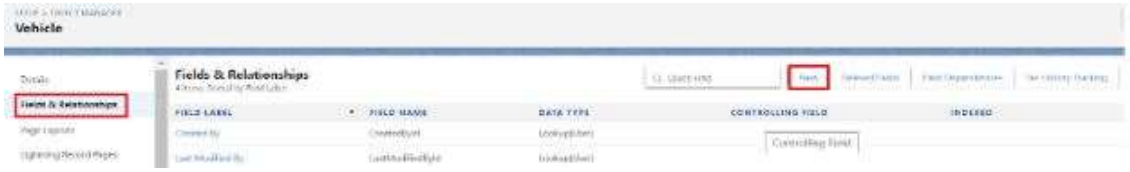
## Step 1: Navigate to the Object

- Go to Setup
- Open Object Manager
- Type `Vehicle` in the Quick Find bar
- Click on `Vehicle__c`




**Step 2: Create Each Field**

Click `New` under `Fields & Relationships` and repeat for each field below:




## **Field 1: Vehicle Name**

- Data Type: Text
- Field Label: Vehicle Name
- Length: 80
- Settings: Mark as Required if every vehicle must have a name
- Field-Level Security: Keep Visible for all profiles

- Page Layout: Add to your primary layout
- Click Save & New

**Field 2: Vehicle Model**
- Data Type: Picklist
- Field Label: Vehicle Model
- Picklist Values: Enter each on a new line: Sedan, SUV, EV, Truck, Hatchback
- Settings: Check "Sort values alphabetically" and set a default if needed
- Field-Level Security: Keep Visible for all profiles
- Page Layout: Add to your primary layout
- Click Save & New



**Field 3: Stock Quantity**
- Data Type: Number
- Field Label: Stock Quantity
- Length: 18
- Decimal Places: 0
- Settings: Set minimum value to 0 to prevent negative inventory
- Field-Level Security: Keep Visible for all profiles
- Page Layout: Add to your primary layout
- Click Save & New

**Field 4: Price**
- Data Type: Currency
- Field Label: Price
- Length: 16
- Decimal Places: 2
- Settings: Leave currency type as Organization Default

- Field-Level Security: Keep Visible for all profiles
- Page Layout: Add to your primary layout
- Click Save & New

**Field 5: Dealer**
- Data Type: Lookup Relationship
- Related To: Vehicle Dealer
- Field Label: Dealer
- Settings: Allow clearing of lookup value, keep lookup dialog enabled
- Field-Level Security: Keep Visible for all profiles
- Page Layout: Add to your primary layout
- Click Save & New

**Field 6: Status**
- Data Type: Picklist
- Field Label: Status
- Picklist Values: Enter each on a new line: Available, Out of Stock, Discontinued
- Settings: Mark as Required if status tracking is mandatory
- Field-Level Security: Keep Visible for all profiles
- Page Layout: Add to your primary layout
- Click Save




## Step 3: Verify Setup
- Open the App Launcher and select your WhatNext Vision Motors app
- Click the Vehicles tab
- Click New and confirm all six fields appear on the creation page
- Test the picklist dropdowns, lookup search, and any required field validations

Reply with `Continue` if you want the exact same plain-text steps for the remaining objects (Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, and Vehicle Service Request), or let me know if you need validation rules, record types, or page layout adjustments next.

Understood. Here are the clear, step-by-step instructions to create the **Stock Quantity** field on the **Vehicle** object, in plain text format without tables.

**Step 1: Navigate to the Object**
- Go to Setup
- Open Object Manager
- Type `Vehicle` in the Quick Find bar
- Click on `Vehicle__c`



### Step 2: Start Field Creation
- Click on `Fields & Relationships`
- Click `New`

### Step 3: Select Data Type
- Choose `Number`
- Click `Next`

### Step 4: Configure Field Details
- Field Label: `Stock Quantity` *(Note: Please use "Quantity" not "Quality" to match your data model)*
- API Name: `Stock_Quantity__c` *(auto-populates)*
- Length: `18`
- Decimal Places: `0` *(since you cannot have partial vehicles in stock)*
- Default Value: Leave blank or set to `0`
- Required: Optional *(check if every vehicle record must have a stock value at creation)*
- Unique: Unchecked
- External ID: Unchecked
- Click `Next`



### Step 5: Set Field-Level Security
- Ensure `Visible` is checked for all relevant profiles (System Administrator, Sales User, etc.)
- Click `Next`

### Step 6: Add to Page Layouts
- Confirm your primary Vehicle page layout is selected
- This ensures the field appears on record detail pages
- Click `Save & New` to continue creating the next field

Understood. Here are the clear, step-by-step instructions to create a **Lookup Relationship** from the **Vehicle** object to the **Dealer** object, in plain text format without tables.

### **Step 1: Navigate to the Object**
- Go to Setup
- Open Object Manager
- Type `Vehicle` in the Quick Find bar
- Click on `Vehicle__c`

### **Step 2: Start Field Creation**
- Click on `Fields & Relationships`
- Click `New`

**Step 3: Select Data Type**
- Choose `Lookup Relationship`
- Click `Next`




**Step 4: Select Related Object**
- In the "Related To" field, start typing `Dealer`
- Select `Vehicle Dealer` (or `Dealer__c` depending on your exact API name)
- Click `Next`

**Step 5: Configure Field Details**
- Field Label: `Dealer`
- API Name: `Dealer__c` (auto-populates)
- Description: Optional – e.g., "Links this vehicle to its selling dealer"
- Click `Next`

Here are the complete, step-by-step instructions to create all remaining fields across your objects. I have kept this in plain list format with no tables, exactly as requested.

****STANDARD FLOW REMINDER FOR EVERY FIELD****
- Go to Setup → Object Manager → Select Object → Fields & Relationships → New
- Choose Data Type → Next
- Fill Field Details → Next
- Keep Field-Level Security Visible → Next
- Add to Page Layout → Save & New (or Save for the last field)

****OBJECT 1: VEHICLE__C (Remaining Fields)****

**Field: Vehicle Name**
- Data Type: Text
- Field Label: Vehicle Name
- Length: 80
- Required: Checked
- Click Next → Next → Save & New

**Field: Price**
- Data Type: Currency
- Field Label: Price
- Length: 16
- Decimal Places: 2
- Required: Checked
- Click Next → Next → Save & New

**Field: Status**
- Data Type: Picklist
- Field Label: Status
- Picklist Values: Enter each on a new line: Available, Out of Stock, Discontinued
- Default Value: Select Available (optional)
- Required: Checked
- Click Next → Next → Save

**OBJECT 2: VEHICLE DEALER__C**

**Field: Dealer Name**
- Data Type: Text
- Field Label: Dealer Name
- Length: 80
- Required: Checked
- Click Next → Next → Save & New

**Field: Dealer Location**
- Data Type: Text
- Field Label: Dealer Location
- Length: 255
- Required: Unchecked
- Click Next → Next → Save & New

**Field: Dealer Code**
- Data Type: Auto Number
- Field Label: Dealer Code
- Display Format: DLR-{0000}
- Starting Number: 1
- Click Next → Next → Save & New

**Field: Phone**
- Data Type: Phone
- Field Label: Phone
- Click Next → Next → Save & New

**Field: Email**
- Data Type: Email
- Field Label: Email
- Click Next → Next → Save

****OBJECT 3: VEHICLE CUSTOMER__C****

**Field: Customer Name**
- Data Type: Text
- Field Label: Customer Name
- Length: 80
- Required: Checked
- Click Next → Next → Save & New

**Field: Email**
- Data Type: Email
- Field Label: Email
- Click Next → Next → Save & New

**Field: Phone**
- Data Type: Phone
- Field Label: Phone
- Click Next → Next → Save & New

**Field: Address**
- Data Type: Text Area
- Field Label: Address
- Click Next → Next → Save & New

**Field: Preferred Vehicle Type**
- Data Type: Picklist
- Field Label: Preferred Vehicle Type
- Picklist Values: Sedan, SUV, EV, Truck, Hatchback

- Click Next → Next → Save

**OBJECT 4: VEHICLE ORDER__C**

**Field: Customer**
- Data Type: Lookup Relationship
- Related To: Vehicle Customer
- Field Label: Customer
- Child Relationship Name: Orders__r
- Allow users to clear value: Checked
- Click Next → Next → Save & New

**Field: Vehicle**
- Data Type: Lookup Relationship
- Related To: Vehicle
- Field Label: Vehicle
- Child Relationship Name: Orders__r
- Allow users to clear value: Checked
- Click Next → Next → Save & New

**Field: Order Date**
- Data Type: Date
- Field Label: Order Date
- Default Value: Today (optional)
- Required: Checked
- Click Next → Next → Save & New

**Field: Status**
- Data Type: Picklist
- Field Label: Status
- Picklist Values: Pending, Confirmed, Delivered, Canceled
- Default Value: Pending
- Required: Checked
- Click Next → Next → Save

**OBJECT 5: VEHICLE TEST DRIVE__C**

**Field: Customer**
- Data Type: Lookup Relationship

- Related To: Vehicle Customer
- Field Label: Customer
- Child Relationship Name: Test_Drives__r
- Allow users to clear value: Checked
- Click Next → Next → Save & New

**Field: Vehicle**
- Data Type: Lookup Relationship
- Related To: Vehicle
- Field Label: Vehicle
- Child Relationship Name: Test_Drives__r
- Allow users to clear value: Checked
- Click Next → Next → Save & New

**Field: Test Drive Date**
- Data Type: Date
- Field Label: Test Drive Date
- Default Value: Today (optional)
- Required: Checked
- Click Next → Next → Save & New

**Field: Status**
- Data Type: Picklist
- Field Label: Status
- Picklist Values: Scheduled, Completed, Canceled
- Default Value: Scheduled
- Required: Checked
- Click Next → Next → Save

****OBJECT 6: VEHICLE SERVICE REQUEST__C****

**Field: Customer**
- Data Type: Lookup Relationship
- Related To: Vehicle Customer
- Field Label: Customer
- Child Relationship Name: Service_Requests__r
- Allow users to clear value: Checked
- Click Next → Next → Save & New

**Field: Vehicle**
- Data Type: Lookup Relationship
- Related To: Vehicle
- Field Label: Vehicle
- Child Relationship Name: Service_Requests__r
- Allow users to clear value: Checked
- Click Next → Next → Save & New

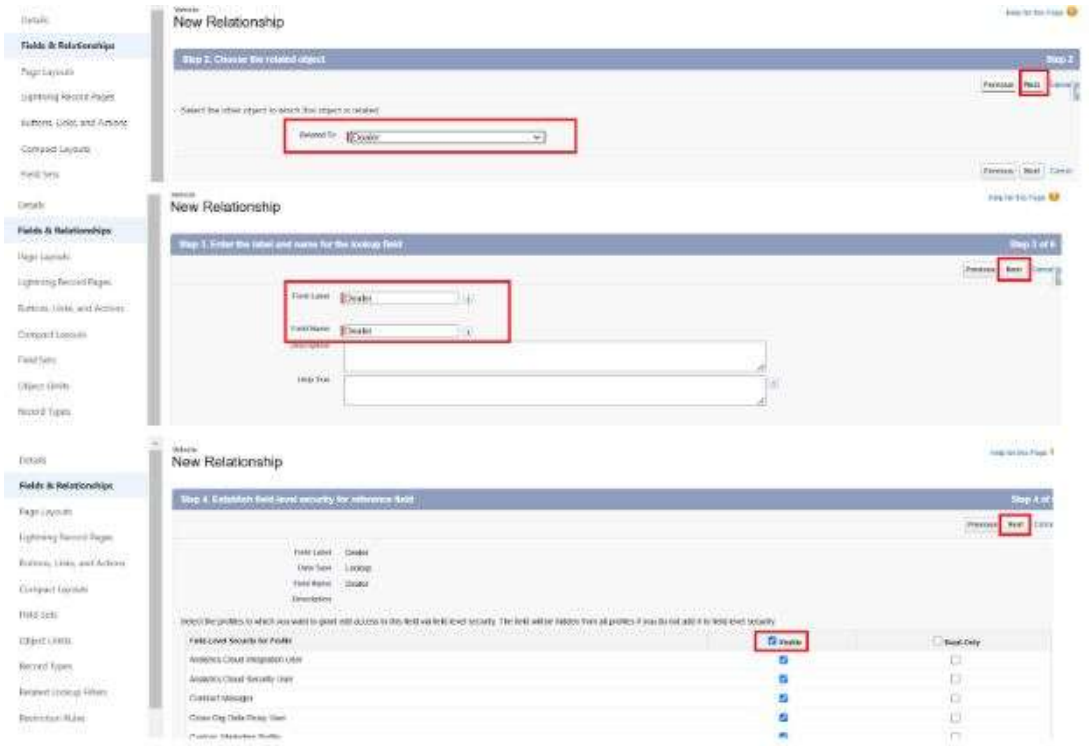
**Field: Service Date**
- Data Type: Date
- Field Label: Service Date
- Default Value: Today (optional)
- Required: Checked
- Click Next → Next → Save & New

**Field: Issue Description**
- Data Type: Text Area
- Field Label: Issue Description
- Length: 500 (or default 255)
- Required: Checked
- Click Next → Next → Save & New

**Field: Status**
- Data Type: Picklist
- Field Label: Status
- Picklist Values: Requested, In Progress, Completed
- Default Value: Requested
- Required: Checked
- Click Next → Next → Save

****FINAL VERIFICATION STEPS****
- Open your WhatNext Vision Motors app
- Click each tab (Vehicle, Dealer, Customer, Order, Test Drive, Service Request)
- Click New on each tab
- Confirm all fields appear, picklists show correct values, lookups search correctly, and required fields block save if empty
- Create one test record per object to verify data saves properly

Reply with `Next` if you want validation rules, record types, or page layout arrangements for these objects, or let me know if you need help fixing any field that did not appear correctly.




**Step 6: Configure Relationship Settings**

- Relationship Name: `Vehicle_Dealer_Rel` (auto-suggested, you can keep or customize)
- Child Relationship Name: `Vehicles__r` (this appears on the Dealer side; keep default unless you have a naming standard)
- Lookup Filter: Optional – you can add filters later if needed (e.g., only show active dealers)
- Allow users to clear the lookup value: ☑ Checked (recommended for flexibility)
- Enable lookup dialog to list available records: ☑ Checked (default)
- Click `Next`

**Step 7: Set Field-Level Security**

- Ensure `Visible` is checked for all profiles that need to see or edit this field (System Administrator, Sales User, etc.)
- Click `Next`

**Step 8: Add to Page Layouts**

- Confirm your primary Vehicle page layout is selected
- This ensures the Dealer lookup field appears on vehicle record detail pages
- Click `Save`

**Step 1: Open Flow Builder**

- Go to Setup
- In Quick Find, type Flows
- Click Flows
- Click New Flow




**Step 2: Choose Flow Type**

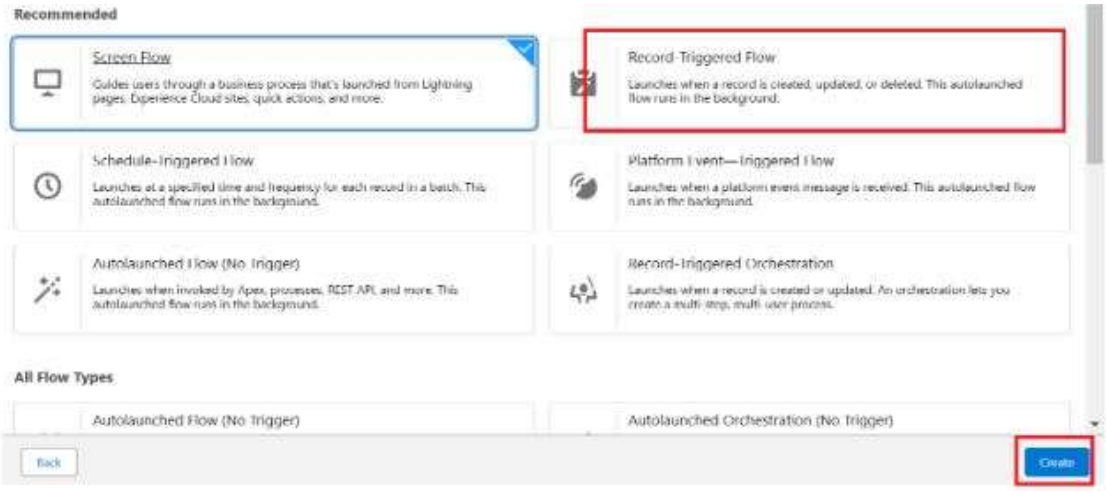
- Select Start From Scratch
- Click Next
- Select Record-Triggered Flow
- Click Create




**Step 3: Configure the Start Element**

- Object: Vehicle Order
- Trigger the Flow When: A record is created
- Set Entry Conditions:
  - Condition Requirements: All Conditions Are Met (AND)
  - Field: Status__c
  - Operator: Equals
  - Value: Pending
- Optimize the Flow For: Actions and Related Records

- Click Done

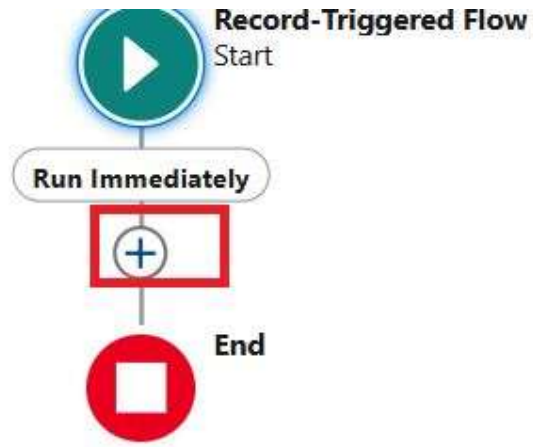



**Step 4: Add First Get Records Element**
- Click the + icon below the Start element
- Select Get Records
- Label: Get Customer Information
- How to Find Records: Use conditions and set field values
- Object: Vehicle Customer
- Condition Requirements: All Conditions Are Met (AND)
- Field: Id
- Operator: Equals
- Value: Click the lightning bolt icon, then select Record Triggering Context → Record → Id
- How Many Records to Store: Only the first record
- How to Store Record Data: Automatically store all fields
- Click Done

```
app = Flask(__name__)
```

**Step 5: Add Second Get Records Element**
- Click the + icon below Get Customer Information
- Select Get Records
- Label: Get Nearest Dealer
- How to Find Records: Use conditions and set field values
- Object: Vehicle Dealer
- Condition Requirements: All Conditions Are Met (AND)

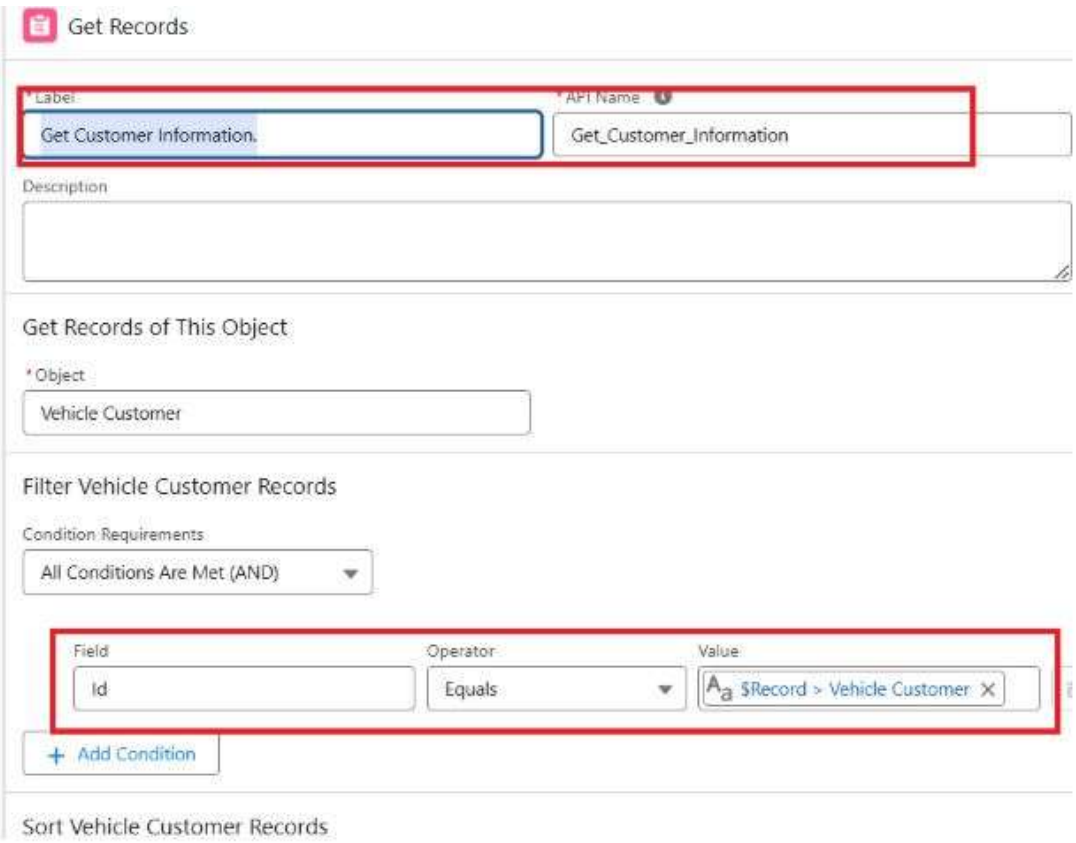
- Field: Dealer_Location__c
- Operator: Equals
- Value: Click the lightning bolt icon, then select Get Customer Information → Address__c
- How Many Records to Store: Only the first record
- How to Store Record Data: Automatically store all fields
- Click Done





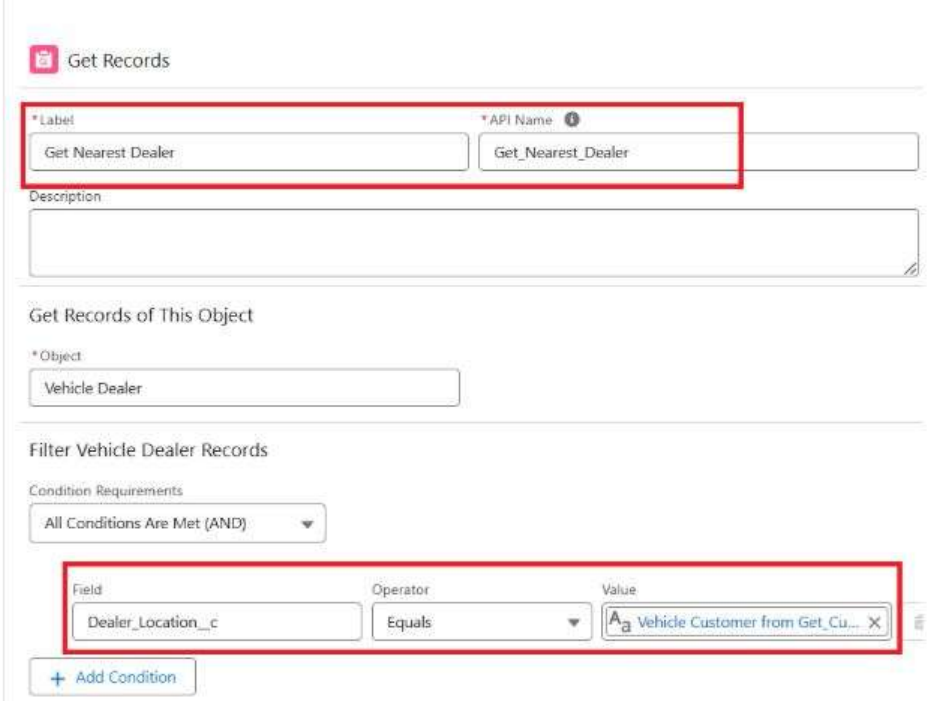


**Step 6: Add Update Records Element (Critical Fix Applied)**
- Click the + icon below Get Nearest Dealer
- Select Update Records
- Label: Assign Dealer to Order
- How to Find Records to Update and Set Their Values: Use the IDs and all field values from a record or record collection
- Record: Click the lightning bolt icon, then select Record Triggering Context → Record (this targets the triggering Vehicle Order)
- Set Field Values for the Vehicle Order Record:
  - Field: Dealer__c (Note: This lookup field must exist on the Vehicle Order object)
  - Value: Click the lightning bolt icon, then select Get Nearest Dealer → Id
- Click Done




**Step 7: Save and Activate**

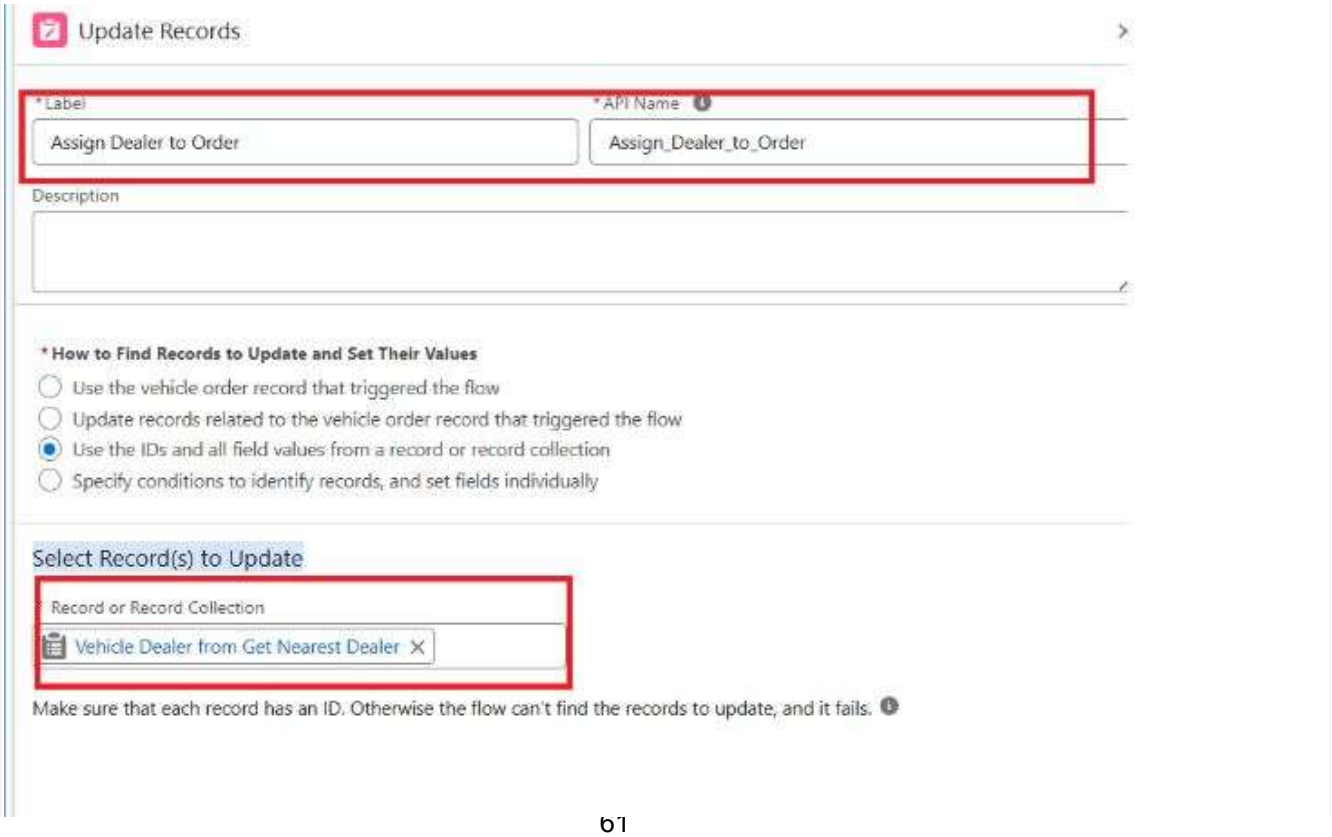
- Click Save
- Label: Auto Assign Dealer
- API Name: Auto_Assign_Dealer (auto-populates)
- Description: Optional
- Click Save
- Click Activate
- Confirm the activation prompt




**Step 8 :** Click **+** → Select **Update Records**

**Label :** Assign Dealer to Order

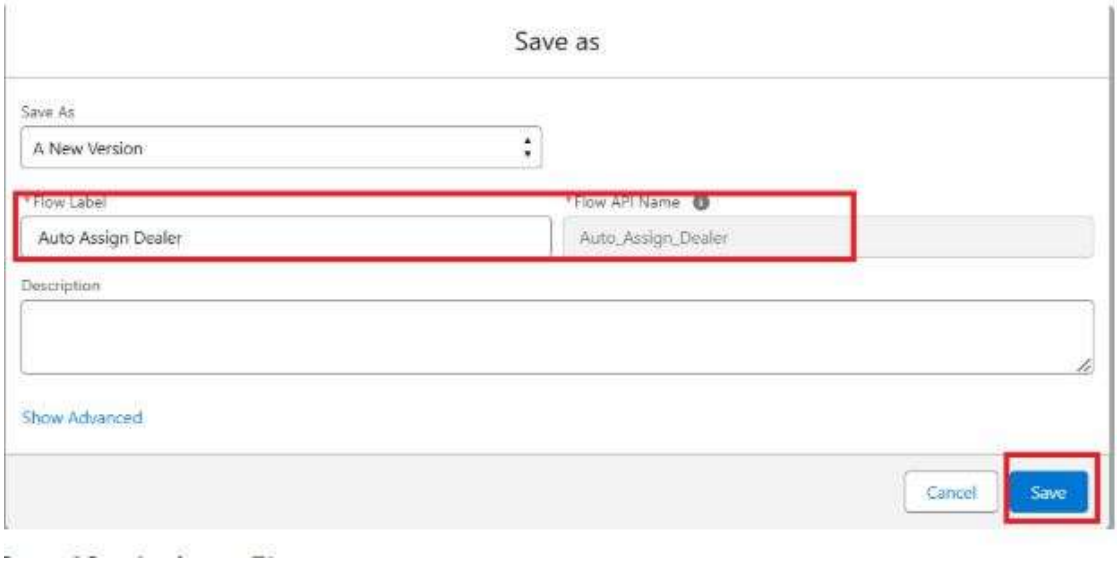
*How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection



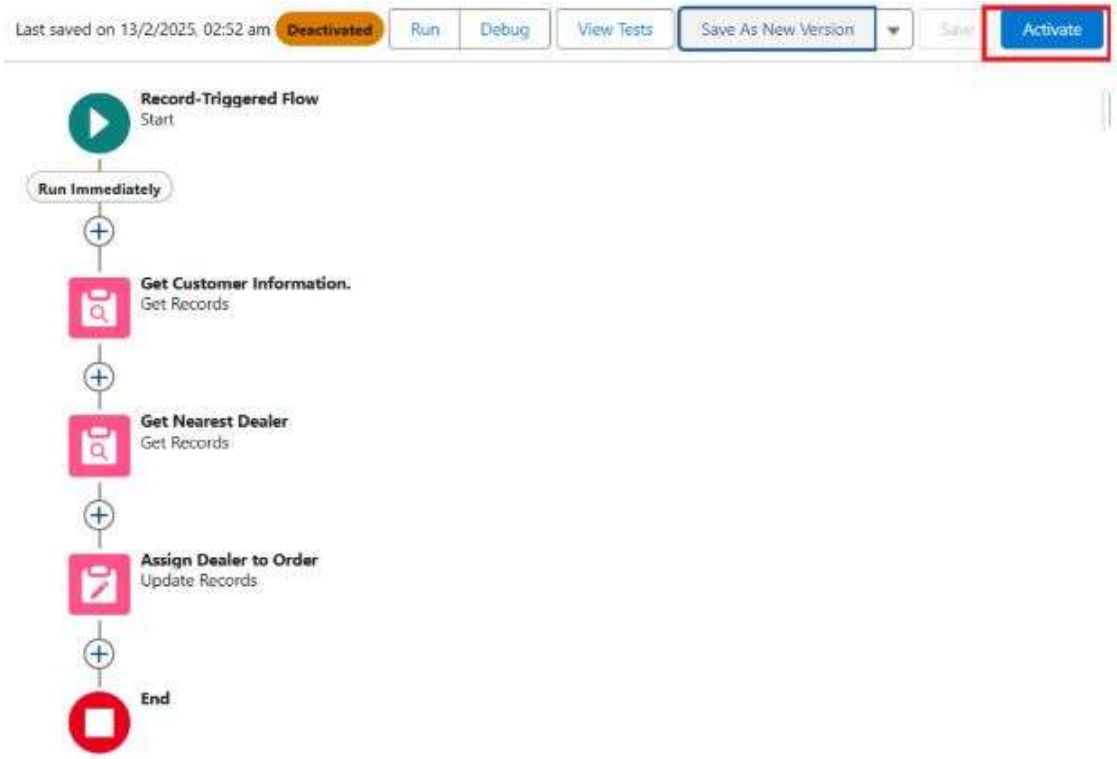
Select Record(s) to Update : {!Get_Nearest_Dealer}

**Step 9 :** Click Save and Give label Name and Activate Flow.

Label Name : Auto Assign Dealer

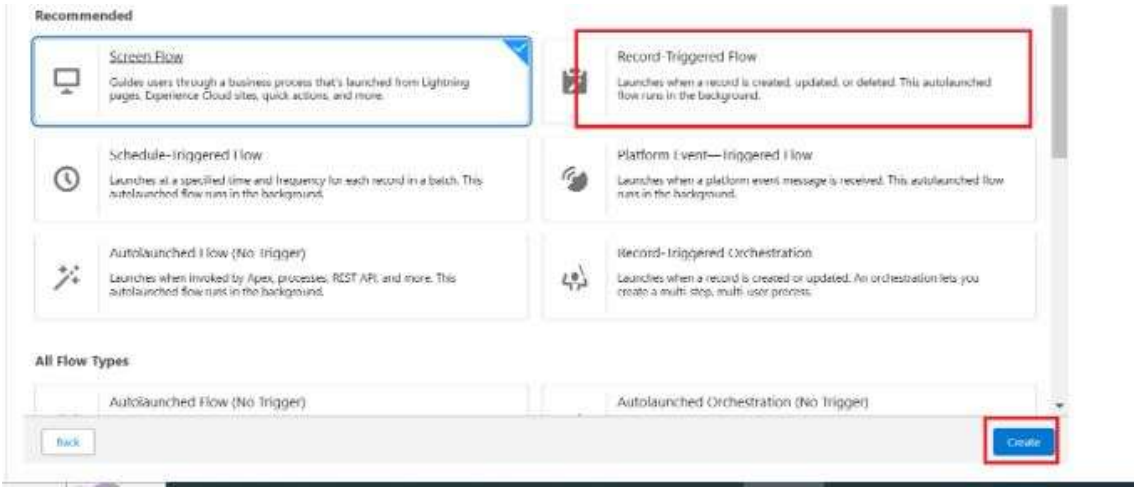



**Step 10 :** Activate Flow



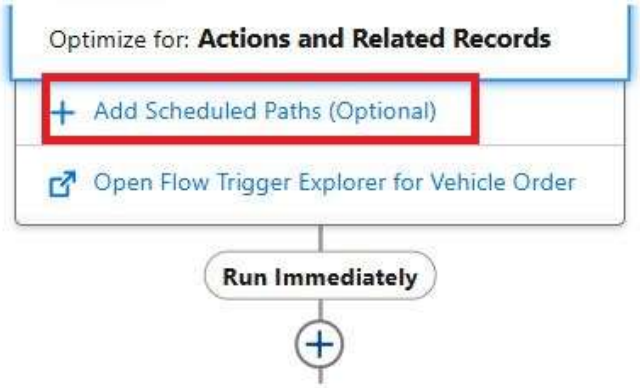
**Step 1: Open Flow Builder**

- Go to Setup
- In Quick Find, type Flows
- Click Flows
- Click New Flow
- Select Record-Triggered Flow
- Click Create




**Step 2: Configure the Start Element**

- Object: Vehicle Test Drive
- Trigger the Flow When: A record is created or updated
- Set Entry Conditions:
  - Condition Requirements: All Conditions Are Met (AND)
  - Field: Status__c
  - Operator: Equals
  - Value: Scheduled
- Optimize the Flow For: Actions and Related Records
- Click Done







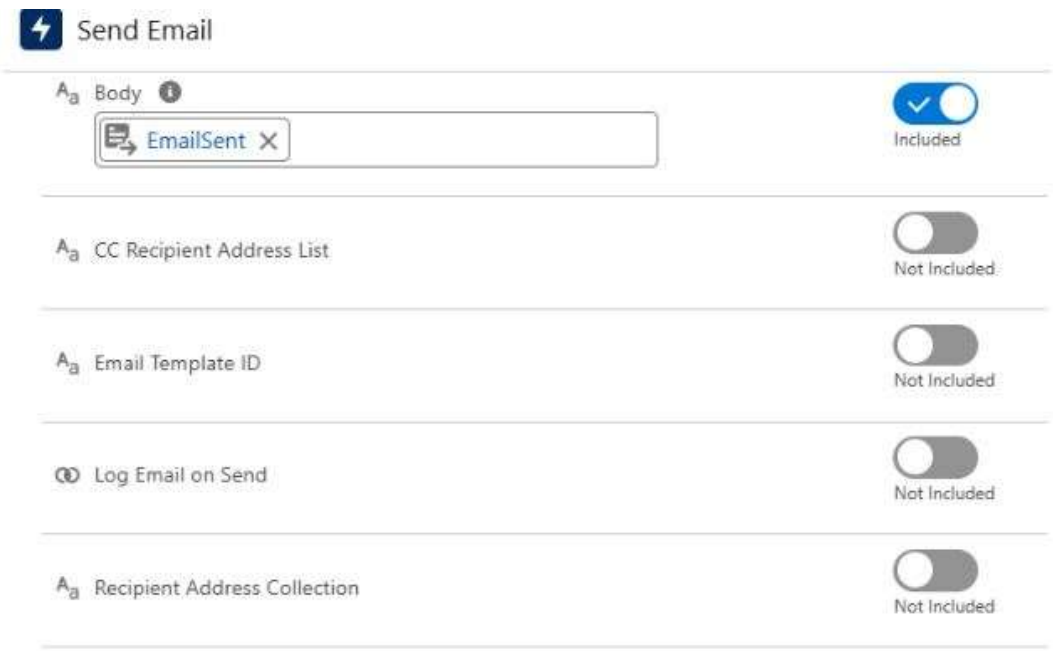
**Step 3: Add Scheduled Path**

- Click the + icon below the Start element
- Select Add Scheduled Paths
- Label: Reminder Before Test Drive
- Time Source: Test_Drive_Date__c
- Offset Number: 1
- Offset Options: Days Before
- Click Done

**Step 4: Get Customer Information**

- Inside the Scheduled Path, click the + icon
- Select Get Records
- Label: Get Customer Information
- How to Find Records: Use conditions and set field values

- Object: Vehicle_Customer__c
- Condition Requirements: All Conditions Are Met (AND)
- Field: Id
- Operator: Equals
- Value: Click the lightning bolt icon, then select Record Triggering Context → Record → Customer__c
- How Many Records to Store: Only the first record
- How to Store Record Data: Automatically store all fields
- Click Done

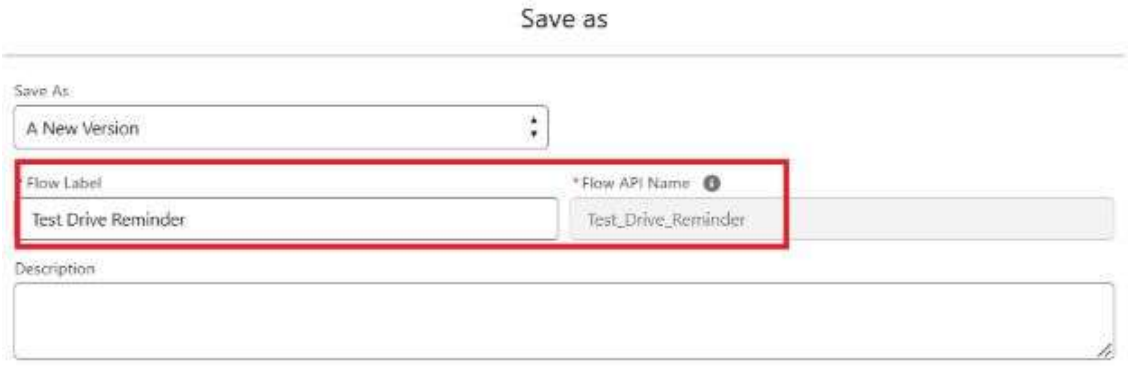



**Step 5: Create Email Body**
- Click the + icon below Get Customer Information
- Select Text Template
- Label: Test Drive Email Body
- API Name: Test_Drive_Email_Body (auto-populates)
- In the template editor, enter the following message:
  Hello {!Get_Customer_Information.Customer_Name__c},
  This is a friendly reminder that your test drive for the {!$Record.Vehicle__r.Name} is scheduled for tomorrow. Please arrive 10 minutes early at your preferred location.

- Click Done

**Step 6: Configure Send Email Action**
- Click the + icon below the Text Template

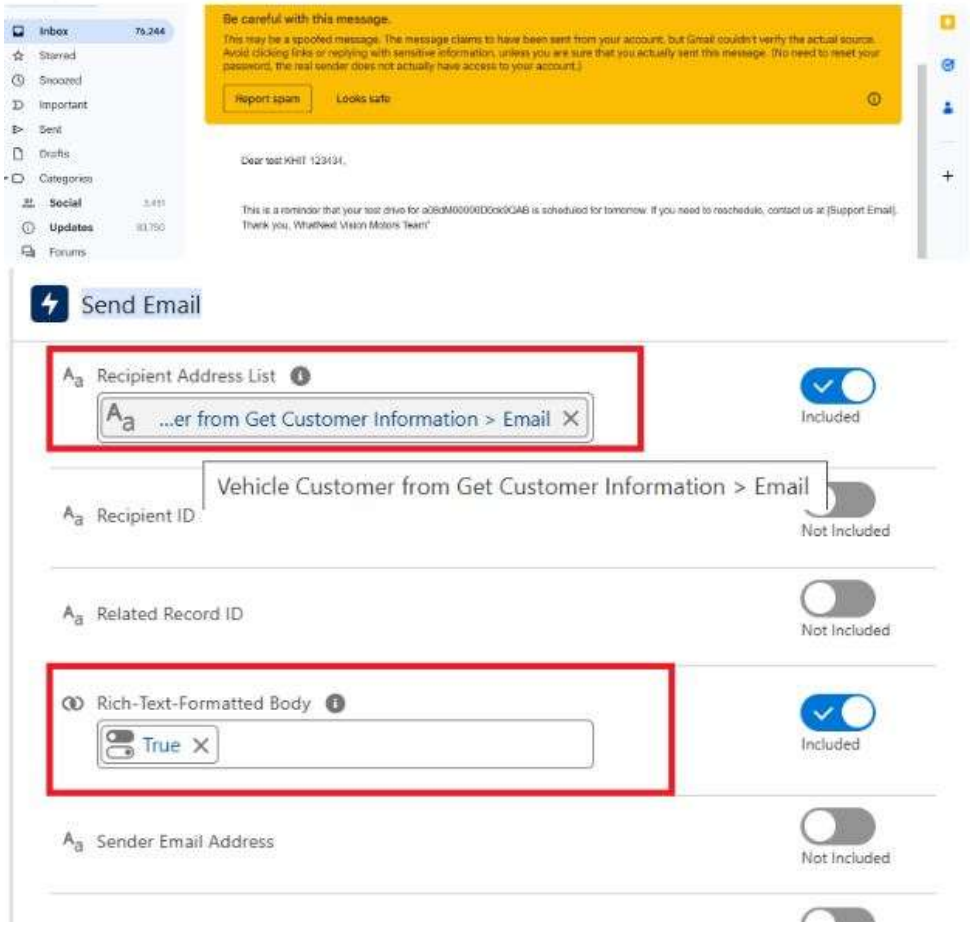
- Select Action
- Action Type: Send Email
- Label: Send Test Drive Reminder
- Sender Type: Default Workflow User or Org-Wide Email Address (choose one)
- Recipient Type: Email Address
- Recipient Address List: Click the lightning bolt icon, then select Get Customer Information → Email__c
- Subject: Reminder: Your Test Drive is Tomorrow!
- Body: Click the lightning bolt icon, then select Test Drive Email Body
- Click Done




**Step 7: Save and Activate

- Click Save
- Label: Test Drive Reminder
- API Name: Test_Drive_Reminder (auto-populates)
- Description: Sends an email reminder one day before a scheduled test drive
- Click Save
- Click Activate
- Confirm the activation prompt

**Test your Flow :**




**How to Test the Flow**

- Scheduled paths do not run immediately. They queue and execute at the exact offset time.
- Create a Vehicle Customer record with a valid email address.
- Create a Vehicle Test Drive record:
  - Link it to the test Customer
  - Link it to any Vehicle
  - Set Status to Scheduled
  - Set Test Drive Date to exactly 1 day from today (e.g., if today is April 8, set it to April 9)
- Save the Test Drive record.
- Open Setup → Flows → Click "Test Drive Reminder" → Click the "View Scheduled Interviews" button (or check Email Logs / Flow Interviews)
- Wait for the scheduled time to pass (Salesforce may take up to 15 minutes past the exact time to process)
- Check the customer email inbox for the reminder message

Here are the corrected, production-ready Apex classes, trigger, and batch job for your Vehicle Order management system. All instructions and code are in plain text format without tables, as requested.

**STEP 1: OPEN DEVELOPER CONSOLE**
- Click the Gear icon in Salesforce
- Select Developer Console

**STEP 2: CREATE THE TRIGGER HANDLER CLASS**
- Click File → New → Apex Class
- Class Name: VehicleOrderTriggerHandler
- Paste the following corrected code:

```apex
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(
        List<Vehicle_Order__c> newOrders,
        Map<Id, Vehicle_Order__c> oldOrders,
        Boolean isBefore,
        Boolean isAfter,
        Boolean isInsert,
        Boolean isUpdate) {

        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders, oldOrders, isUpdate);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders, oldOrders, isUpdate);
            }
        }
    }

    private static void preventOrderIfOutOfStock(
        List<Vehicle_Order__c> orders,
        Map<Id, Vehicle_Order__c> oldOrders,
        Boolean isUpdate) {
```

```
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null) {
            if (isUpdate) {
                Vehicle_Order__c oldOrder = oldOrders.get(order.Id);
                if (oldOrder != null && oldOrder.Status__c == order.Status__c) {
                    continue;
                }
            }
            vehicleIds.add(order.Vehicle__c);
        }
    }

    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
            [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
        );

        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c <= 0 && order.Status__c == 'Confirmed') {
                    order.addError('This vehicle is out of stock. Order cannot be confirmed.');
                }
            }
        }
    }
}

private static void updateStockOnOrderPlacement(
    List<Vehicle_Order__c> orders,
    Map<Id, Vehicle_Order__c> oldOrders,
    Boolean isUpdate) {

    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            if (isUpdate) {
                Vehicle_Order__c oldOrder = oldOrders.get(order.Id);
```

```
            if (oldOrder != null && oldOrder.Status__c == 'Confirmed') {
                continue;
            }
        }
        vehicleIds.add(order.Vehicle__c);
    }
}

if (!vehicleIds.isEmpty()) {
    Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
        [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
    );

    List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
    for (Vehicle_Order__c order : orders) {
        if (vehicleStockMap.containsKey(order.Vehicle__c)) {
            Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
            if (vehicle.Stock_Quantity__c > 0) {
                vehicle.Stock_Quantity__c -= 1;
                vehiclesToUpdate.add(vehicle);
            }
        }
    }

    if (!vehiclesToUpdate.isEmpty()) {
        update vehiclesToUpdate;
    }
}
}
}
```

- Click Save

**STEP 3: CREATE THE TRIGGER**
- Click File → New → Apex Trigger
- Name: VehicleOrderTrigger
- Object: Vehicle_Order__c
- Paste the following code:

```apex
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {
    VehicleOrderTriggerHandler.handleTrigger(
        Trigger.new,
        Trigger.oldMap,
        Trigger.isBefore,
        Trigger.isAfter,
        Trigger.isInsert,
        Trigger.isUpdate
    );
}
```
- Click Save

**STEP 4: CREATE THE BATCH CLASS**
- Click File → New → Apex Class
- Class Name: VehicleOrderBatch
- Paste the following corrected code:

```apex
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }
```

```
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
                [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
            );

            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

            for (Vehicle_Order__c order : orderList) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        order.Status__c = 'Confirmed';
                        vehicle.Stock_Quantity__c -= 1;
                        ordersToUpdate.add(order);
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }

            if (!ordersToUpdate.isEmpty()) {
                update ordersToUpdate;
            }
            if (!vehiclesToUpdate.isEmpty()) {
                update vehiclesToUpdate;
            }
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
    }
}
```

- Click Save

**STEP 5: CREATE THE SCHEDULER CLASS**

- Click File → New → Apex Class

- Class Name: VehicleOrderBatchScheduler
- Paste the following code:

```apex
global class VehicleOrderBatchScheduler implements Schedulable {

    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50);
    }
}
```

- Click Save

**STEP 6: SCHEDULE THE BATCH JOB**

- In Developer Console, click Debug → Open Execute Anonymous Window
- Paste the following code to schedule daily at midnight:

```apex
String cronExp = '0 0 0 * * ?';
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

- Click Execute

**STEP 7: VERIFY SCHEDULED JOB**

- Go to Setup → Quick Find → Scheduled Jobs
- Confirm "Daily Vehicle Order Processing" appears with next fire time at 12:00 AM

**STEP 8: TEST THE IMPLEMENTATION**

Test Scenario 1: Prevent Order When Out of Stock

- Create a Vehicle record with Stock_Quantity__c = 0
- Create a Vehicle_Order__c record linked to that vehicle with Status__c = Confirmed
- Save should fail with error: "This vehicle is out of stock. Order cannot be confirmed."

Test Scenario 2: Auto-Decrement Stock on Confirm

- Create a Vehicle record with Stock_Quantity__c = 5
- Create a Vehicle_Order__c record linked to that vehicle with Status__c = Confirmed
- Save the order

- Reopen the Vehicle record and confirm Stock_Quantity__c = 4

### Test Scenario 3: Batch Job Converts Pending to Confirmed

- Create a Vehicle record with Stock_Quantity__c = 10
- Create a Vehicle_Order__c record with Status__c = Pending linked to that vehicle
- Wait for the scheduled batch to run at midnight OR run manually via Execute Anonymous:

```apex
Database.executeBatch(new VehicleOrderBatch(), 50);
```

- After execution, confirm the order Status__c changed to Confirmed and vehicle stock decreased by 1

Here are **advantages and disadvantages** of *WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence* (based on your project concept + future mobility trends):

## ☑ Advantages

**1. Improved Efficiency through Automation**

- Use of Salesforce automation (flows, AI, Apex) reduces manual work.
- Faster processes like vehicle ordering and approvals. (Scribd)

**2. Enhanced Customer Experience**

- CRM system improves interaction, personalization, and service quality.
- AI-driven insights help understand customer needs better. (McKinsey & Company)

**3. Data-Driven Decision Making**

- Integration with Data Cloud enables real-time analytics.
- Helps in better planning, forecasting, and management.

**4. Innovation with AI & Smart Technologies**

- Use of AI, NLP, and automation improves productivity and accuracy.
- Supports intelligent services and predictive systems. (McKinsey & Company)

**5. Scalability and Flexibility**

- Salesforce platform allows easy expansion and customization.
- Suitable for growing automotive and mobility businesses.

**6. Sustainable and Future-Oriented Mobility**

- Aligns with trends like electric vehicles, smart transport, and digital mobility.
- Contributes to eco-friendly and efficient transportation systems. (Zeiss)

## ❌ Disadvantages

**1. High Implementation Cost**

- Developing CRM systems, AI integration, and automation tools can be expensive.

- Requires investment in software, infrastructure, and training.

**2. Technical Complexity**

- Integration of AI, cloud, and automation systems is complex.
- Needs skilled developers and continuous maintenance. (TÜV SÜD)

**3. Data Security & Privacy Risks**

- Handling large customer data increases risk of cyber threats.
- Strong security measures are required. (TÜV SÜD)

**4. Dependency on Technology**

- System failures or downtime can affect operations.
- Heavy reliance on cloud platforms.

**5. Training Requirement**

- Employees need training to use Salesforce tools effectively.
- Learning curve can slow initial adoption.

**6. Regulatory & Compliance Challenges**

- Must follow automotive and data protection regulations.
- Compliance can be complex and time-consuming. (TÜV SÜD)

**Conclusion**

WhatNext Vision Motors represents a forward-thinking approach to the future of mobility by combining innovation, sustainability, and advanced technology. By leveraging the power of Salesforce, the company has successfully streamlined its operations, improved customer relationship management, and enhanced overall efficiency.

The integration of data management, automation, and Apex development enables smarter decision-making and scalable solutions. This project highlights how digital transformation can drive growth in the automotive industry while delivering a superior customer experience.

In conclusion, WhatNext Vision Motors is well-positioned to lead the next generation of smart, connected, and sustainable transportation.

Here are **Future Scope** and **References / Appendices** for your project *"WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence"*:

## 🚀 Future Scope

**1. Expansion of Electric Mobility (EVs)**

- Future focus on electric vehicles, hybrid systems, and sustainable transport.
- EVs may account for a significant share of vehicle sales by 2030, showing strong growth potential. (McKinsey & Company)

**2. Integration of AI and Automation**

- AI will play a major role in predictive maintenance, smart driving, and customer analytics.
- Helps in improving safety, efficiency, and decision-making processes. (S&P Global)

**3. Autonomous and Smart Vehicles**

- Development of self-driving cars and advanced driver-assistance systems (ADAS).
- Future mobility will rely on automation and real-time decision systems.

**4. Connected Mobility Ecosystem**

- Vehicles will become part of a connected network (IoT + Cloud).
- Enables real-time tracking, navigation, and enhanced user experience. (McKinsey & Company)

**5. Mobility as a Service (MaaS)**

- Shift from vehicle ownership to shared mobility services.
- Platforms can integrate booking, ride-sharing, and subscription models.

**6. Data-Driven Business Growth**

- Use of CRM tools like Salesforce for analytics and personalization.
- Data will drive marketing, sales, and customer retention strategies.

**7. Sustainable and Green Transportation**

- Focus on reducing carbon emissions and eco-friendly solutions.
- Integration of renewable energy and smart charging infrastructure.

**8. Global Market Expansion**

- Opportunity to expand into international markets with scalable cloud platforms.
- Collaboration with smart cities and transport systems.

## References


1. McKinsey & Company – *Future of Mobility in India*
2. McKinsey Center for Future Mobility – Mobility Trends & Insights
3. S&P Global Mobility – AI in Automotive Industry
4. Research Papers on AI and Electric Vehicles (arXiv publications)
5. Automotive Industry Reports on Electrification & Smart Mobility


(These sources discuss trends like electrification, AI adoption, and connected mobility shaping the future.) (McKinsey & Company)

## Appendices

**Appendix A: System Architecture**

- Salesforce CRM architecture diagram
- Data flow between users, system, and database

**Appendix B: Data Model**

- Objects: Customer, Vehicle, Order, Service
- Relationships between objects

**Appendix C: Automation Details**

- Workflow rules
- Process Builder / Flow diagrams

**Appendix D: Apex & Batch Classes**

- Sample Apex classes for automation
- Batch processing logic for large data handling

**Appendix E: Screenshots**

- Salesforce dashboard
- App interface
- Reports and analytics

**Appendix F: Use Case Diagrams**

- Customer interaction flow
- Admin and system processes